POLITECHNIKA
LUBELSKA
WYDZIAŁ ELEKTROTECHNIKI
I INFORMATYKI

Praca dyplomowa inżynierska

na kierunku Informatyka
w specjalności Inżynieria Oprogramowania

System informatyczny wspomagający stację kontroli pojazdów

An IT system for supporting a vehicle inspection centre

Piotr Bednarski
097566

Tomasz Brzozowski
097572

Promotor: dr Mariusz Dzieńkowski

Lublin 2025

Streszczenie

W niniejszej pracy inżynierskiej zostały przedstawione kolejne etapy projektowania oraz implementacji systemu informatycznego wspomagającego pracę stacji kontroli pojazdów. Prezentowany system jest aplikacją internetową umożliwiająca użytkownikowi zarejestrowanie się na wybrany przez niego termin badania technicznego. Aplikacja została wykonana na stosie technologicznym PostgreSQL, Spring Boot, React oraz skonteneryzowana w technologii Docker. Praca obejmuje m.in. zagadnienia bezstanowej autoryzacji użytkowników z podziałem na role pełnione w systemie przy użyciu Spring Security za pomocą tokenów JWT, problematykę wystawiania API w architekturze REST, kwestię łączenia się z REST API przez warstwę kliencką oraz tematem warstwowej budowy systemów informatycznych.

Słowa kluczowe: system wspomagający pracę stacji kontroli pojazdów, rezerwacja terminu badania technicznego, zarządzanie kalendarzem pracy, aplikacja internetowa, architektura REST, Spring Boot, React, PostgreSQL

Abstract

This engineering thesis presents the successive stages of the design and implementation of an IT system supporting the work of a vehicle inspection centre. The presented system is a web application that allows users to book an appointment of their choice for a technical inspection. The application was developed using the PostgreSQL technology stack, Spring Boot, React and containerised in Docker technology. The thesis covers, among others, the issues of stateless user authentication with different roles in the system using Spring Security via JWT tokens, the problem of issuing APIs in a REST architecture, the issue of connecting to REST APIs via the client layer and the topic of layered construction of information systems.

Keywords: vehicle inspection centre support system, technical inspection appointment booking, work calendar management, web application, REST architecture, Spring Boot, React, PostgreSQL.

Spis treści

1. Wstęp 5
1.1. Wprowadzenie 5
1.2. Analiza rynku pod względem produktów analogicznych 7
2. Cel i zakres pracy 11
2.1. Cel pracy 11
2.2. Zakres pracy 11
2.3. Podział pracy 12
3. Projekt systemu 14
3.1. Wymagania funkcjonalne 14
3.2. Wymagania niefunkcjonalne 16
3.3. Diagramy procesów biznesowych 16
3.4. Diagramy przypadków użycia (DPU) 20
3.5. Diagramy sekwencji 23
3.6. Diagramy aktywności 25
3.7. Diagram ERD bazy danych 27
3.8. Projekt interfejsu 28
4. Implementacja systemu 36
4.1. Użyte technologie 36
4.2. Warstwa serwerowa 38
4.3. Warstwa kliencka 42
4.4. Konteneryzacja 44
4.5. Funkcjonalność rejestracji konta klienta 46
4.6. Funkcjonalność zarządzania pojazdami użytkownika 48
4.7. Funkcjonalność zmiany hasła konta w systemie 50
4.8. Funkcjonalność umawiania się na badanie techniczne 51
4.9. Funkcjonalność odwoływania rezerwacji 55
4.10. Funkcjonalność panelu administratora: Zarządzanie kalendarzem pracy 57
5. Testy aplikacji 60
5.1. Podział testów ze względu na przeznaczenie 60
5.2. Testy warstwy serwera projektowanego systemu 61
6. Wnioski 66
Bibliografia 67

1. Wstęp

1.1. Wprowadzenie

Posiadanie samochodu w Polsce w latach 80-tych i 90-tych nie było jeszcze rzeczą powszechną. Dopiero po przemianie gospodarczej z gospodarki centralnie planowanej na gospodarkę zdecentralizowaną zauważyć można było trend wzrostowy ilości samochodów per capita. Posiadanie samochodu zaczęło z czasem stawać się popularne oraz szerzej dostępne dla przeciętnego mieszkańca. Pozostałością poprzedniego systemu jest kwestia stałej konieczności badania stanu technicznego pojazdów dopuszczonych do ruchu drogowego poprzez państwowe instytucje zwane stacjami kontroli pojazdów. Wraz z rosnącą liczbą samochodów w kraju proporcjonalnie wzrastało zapotrzebowanie na tę usługę. Bezpośrednia konieczność regularnego badania stanu technicznego pojazdów wynika z Art. 81 Ustawy Prawo o ruchu drogowym (Dz. U. 1997 nr 98 poz. 602) z póź. zm [13]. Podobne przepisy obowiązywały już dużo wcześniej. Przykładem może być Art. 63 uchylonej już Ustawy z dnia 1 lutego 1983 r. Prawo o ruchu drogowym (Dz.U. 1983 nr 6 poz. 35) [12]. Można stwierdzić, że badania techniczne pojazdów silnie ugruntowały się w polskiej motoryzacji.

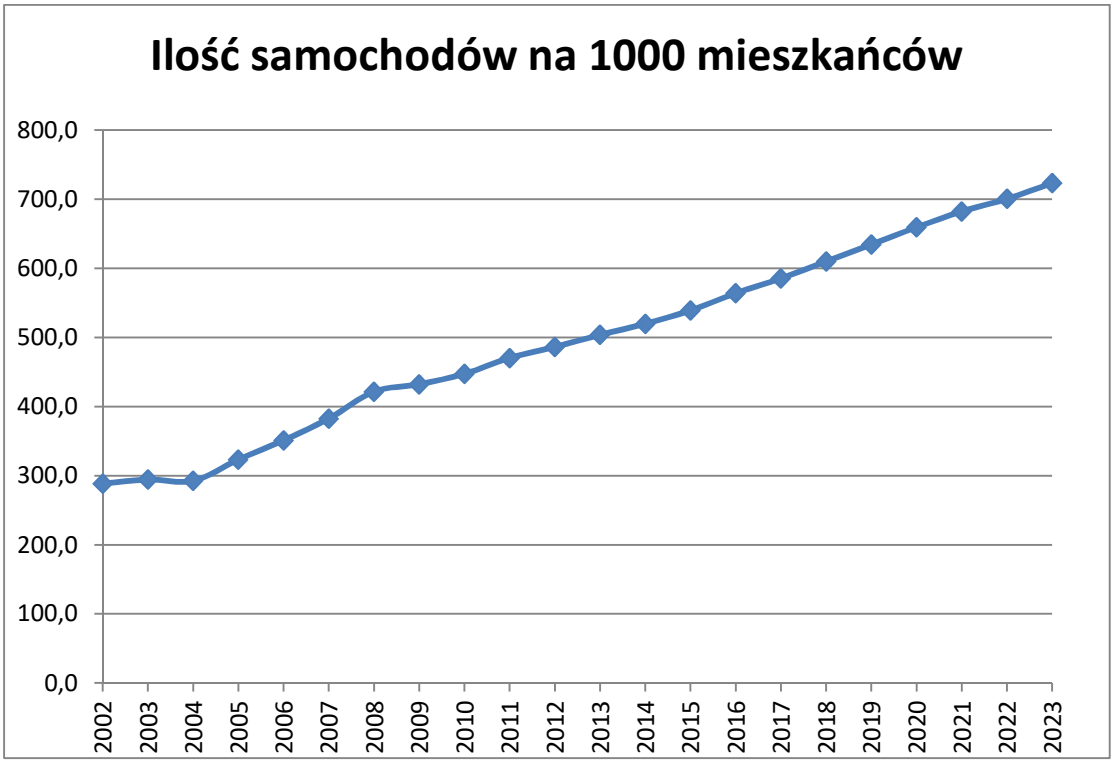
Kolejnym ważnym aspektem jest wysokość opłaty za przeprowadzenie badania technicznego, którą reguluje Rozporządzenie Ministra Infrastruktury z dnia 29 września 2004 r. [10]. w sprawie wysokości opłat związanych z prowadzeniem stacji kontroli pojazdów oraz przeprowadzaniem badań technicznych pojazdów. Bezpośrednia ingerencja państwa w koszt wykonania tej usługi to relikt poprzedniego systemu gospodarki centralnie planowanej.

Według danych z Centralnej Ewidencji Pojazdów i Kierowców obecnie (dane na 2022 r.) co roku wykonuje się około 20 milionów okresowych badań technicznych w ponad 5 tysiącach okręgowych i podstawowych stacjach kontroli pojazdów. Zakładając, że przeważająca ilość kontrolowanych pojazdów to samochody osobowe, można przyjąć (zaniżając obrót na polskim rynku badań technicznych), że koszt pojedynczego badania technicznego to 100 zł. Mając te informacje możemy oszacować, że coroczny kapitał na polskim rynku badań technicznych wynosi co najmniej 2 miliardy złotych.

Ponadto ilość samochodów w Polsce przypadających na 1000 mieszkańców (dane z Głównego Urzędu Statystycznego z lat 2002-2023) [8] z roku na rok rośnie w postępie

liniowym co obrazuje Rysunek 1. Biorąc pod uwagę ogólną niezmienność przepisów regulujących ten rynek, wzrost popytu wynikający z wzrostu ilości pojazdów w Polsce można dojść do konkluzji, że wskaźnik corocznego kapitału będzie rósł. Dodatkowo można przewidywać zwiększenie ceny badań technicznych w przyszłości ze względu na inflację co jeszcze podniesie oszacowaną wartość polskiego rynku badań technicznych.

Reasumując rynek ten jest ciekawy z punktu widzenia próby wsparcia go za pomocą technologii informatycznych w celu usprawnienia jego działania. Wielkość tego rynku jest na tyle znaczna, że wprowadzenie systemu informatycznego wspomagającego jego funkcjonowanie może być opłacalnym działaniem zarówno dla tego rynku, producentów oprogramowania jak i dla klientów stacji kontroli pojazdów.



Rysunek 1.1. Wykres obrazujący wzrost samochodów per capita w funkcji czasu [8]

Informatyzacja systemu rezerwacji terminu badania technicznego z perspektywy klienta jest bardzo pożądanym aspektem. Możliwość umówienia się na badanie w konkretny dzień oraz godzinę zapewnia klienta, że usługa zostanie przeprowadzona w komfortowym dla niego czasie bez obawy oczekiwania w kolejce za innymi klientami. Ponadto taki system informatyczny optymalizuje również pracę samej stacji pod względem przepustowości usług oraz jej możliwości technicznych i organizacyjnych.

1.2. Analiza rynku pod względem produktów analogicznych

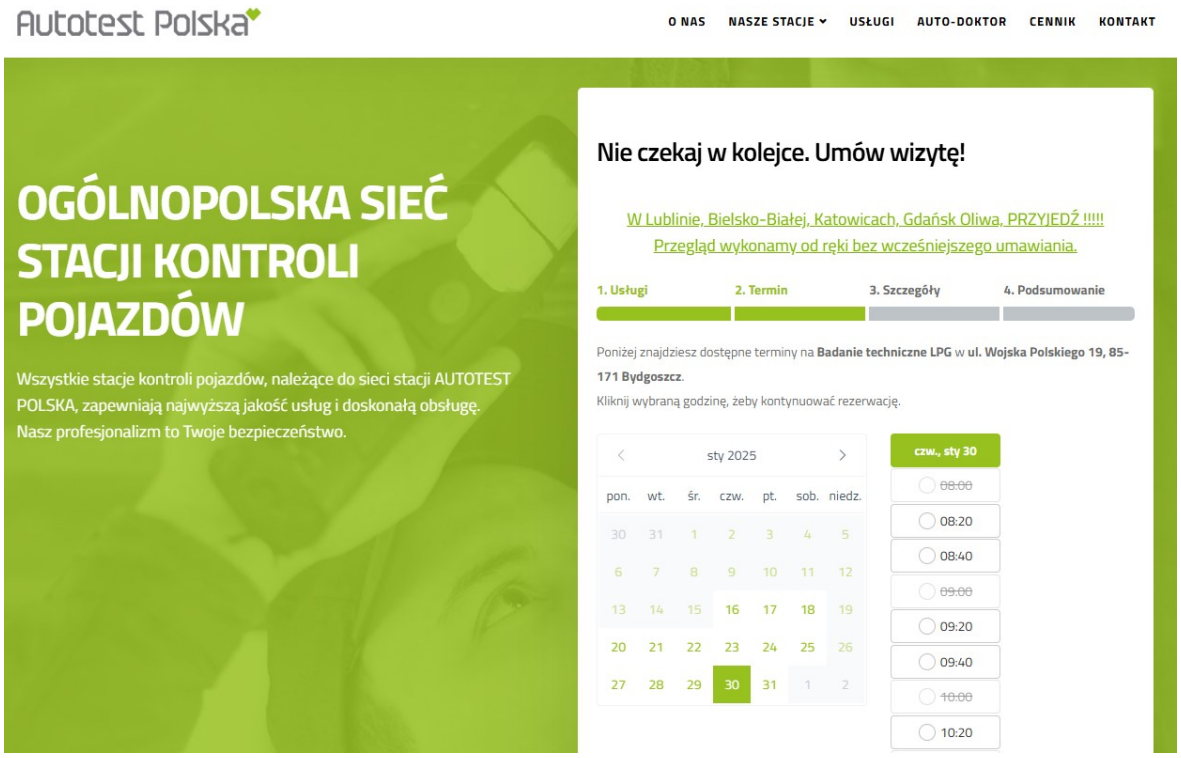
Aplikacja internetowa firmy Euromaster [6]



Rysunek 1.2. Formularz wyboru terminu usługi w serwisie Euromaster [6]

Na rysunku 1.2. zauważyć można znaczną analogię do realizowanej w ramach tej pracy aplikacji. Rozwiązanie firmy Euromaster ma jednak wady. Na początku formularza wybierane są usługi. Niezależnie od ilości wybranych usług czas ich wykonania zawsze jest ustawiony na 1 godzinę. Czas wykonania usługi może mocno się różnić w zależności od typu kontrolowanego samochodu, nie mówiąc o innych usługach, jakie firma Euromaster oferuje. Takie podejście może powodować niechęć do systemu informatycznego zarówno ze strony pracowników jak i klientów oraz generować straty dla firmy.

Aplikacja internetowa firmy Autotest Polska [2]



Rysunek 1.3. Formularz wyboru terminu usługi w serwisie Autotest Polska [2]

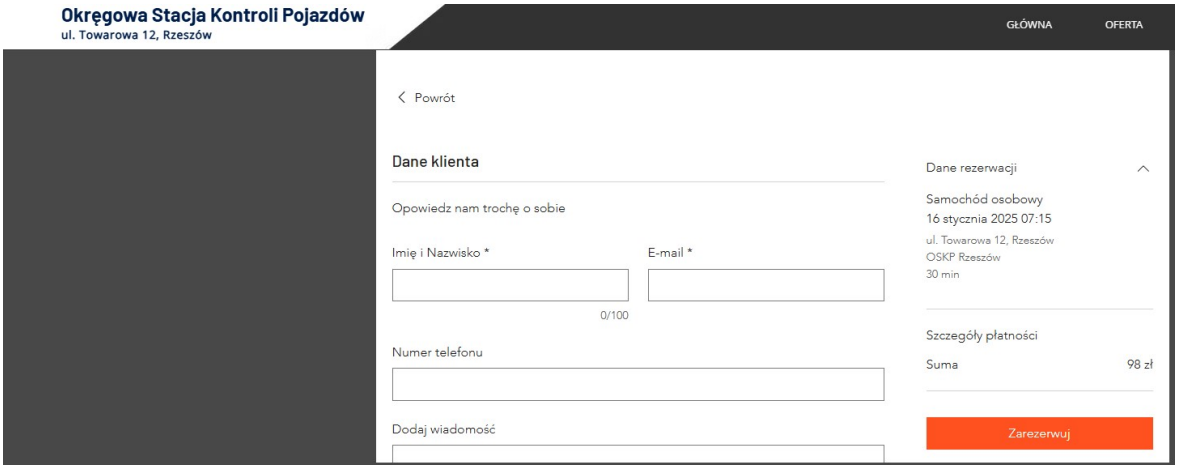
Na rysunku 1.3 można zauważyć, że firma Autotest Polska zdecydowała się na czas przeglądu wynoszący 20 minut niezależnie od specyfiki kontrolowanego pojazdu. To może być zdecydowanie za krótko. Takie podejście może powodować obsunięcia czasowe, co może ostatecznie skutkować utratą zniecierpliwionych klientów. Czas wykonania usługi powinien różnić się w zależności od specyfiki kontrolowanego pojazdu.

Aplikacja internetowa Okręgowej Stacji Kontroli Pojazdów w Rzeszowie [4]



Rysunek 1.4. Formularz wyboru terminu usługi w serwisie Okręgowej Stacji Kontroli Pojazdów w Rzeszowie [4]

Na rysunku 1.4 widać formularz wyboru daty i godziny rezerwowanej usługi. W panelu „Szczegóły usługi" po prawej stronie okna widzimy przewidywany czas trwania usługi oraz koszt. Z punktu widzenia komfortu wyboru dnia i godziny można dojść do wniosku, że należy się na powyższym przykładzie wzorować.



Rysunek 1.5. Formularz wprowadzenia danych klienta umawiającego się na usługę

Rozwiązanie to nie jest jednak pozbawione wad. Za każdą kolejną umawianą usługą należy wybierać typ pojazdu, a także wprowadzać pełnie danych podmiotu, który tę usługę umawia, co prezentuje Rysunek 1.5. Jest to fakt mało uciążliwy z punktu widzenia osób prywatnych, jednakże z punktu widzenia firm posiadających flotę pojazdów poziom

uciążliwości takiego interfejsu może być wyraźnie wyższy. Dużo lepszym pomysłem jest umożliwienie użytkownikom rejestracji prywatnego konta, na którym można przechowywać informacje o danych osobowych, ale także i o posiadanych pojazdach, o ich typie, paliwie (szczególnie w kontekście posiadania instalacji LPG, gdyż jest to czynnik bezpośrednio wpływający na koszt i czas wykonania badania technicznego) oraz terminie wygaśnięcia ważności badania technicznego.

2. Cel i zakres pracy

2.1. Cel pracy

Praca ma na celu zaprojektowanie, implementację i przetestowanie systemu informatycznego wspomagającego pracę stacji kontroli pojazdów (SKP), opartego na stosie technologicznym Spring Boot, React, PostgreSQL oraz konteneryzację w technologii Docker.

2.2. Zakres pracy

Zakresem pracy jest:

- analiza rynku pod kątem podobnych, istniejących aplikacji,
- analiza procesów biznesowych działania SKP,
- zdefiniowanie wymagań funkcjonalnych i niefunkcjonalnych oraz aktorów,
- sporządzenie diagramów BPMN,
- sporządzenie wybranych diagramów UML,
- sporządzenie diagramu ERD bazy danych,
- sporządzenie projektu interfejsu graficznego aplikacji,
- implementacja bazy danych w PostgreSQL,
- implementacja API w architekturze REST w technologii Spring Boot,
- implementacja warstwy klienta w technologii React,
- testy aplikacji,
- konteneryzacja aplikacji w technologii Docker.

2.3. Podział pracy

W poniższej tabeli 2.1 został przedstawiony podział pracy na poszczególnych członków zespołu.

Tabela 2.1. Podział pracy

Zakres pracy		Bednarski Piotr	Brzozowski Tomasz
Wstęp	Wprowadzenie	X	X
	Analiza rynku	X	X
Cel i zakres pracy		X	X
Projekt systemu	Wymagania funkcjonale	X	X
	Wymagania niefunkcjonalne	X	X
	Diagramy BPMN	X	
	Diagramy UML		X
	Diagram ERD bazy danych	X	
	Projekt interfejsu		X
Implementacja systemu	Użyte technologie	X	X
	Implementacja warstwy serwerowej	X	

	Implementacja warstwy klienckiej		X
Zakres pracy		Bednarski Piotr	Brzozowski Tomasz
Implementacja systemu	Konteneryzacja	X	X
	Funkcjonalność rejestracji konta klienta	X	X
	Funkcjonalność zarządzania pojazdami użytkownika	X	X
	Funkcjonalność zmiany hasła konta w systemie	X	X
	Funkcjonalność umawiania się na badanie techniczne	X	X
	Funkcjonalność odwoływania rezerwacji	X	X
	Funkcjonalność panelu administratora	X	X
Testy aplikacji	Podział testów ze względu na przeznaczenie	X	
	Testy warstwy serwera	X	
Wnioski			X
Bibliografia		X	X

3. Projekt systemu

Ten rozdział jest poświęcony opisowi funkcjonalności i założeń tworzonego systemu. Ujęte i omówione zostały funkcje, jakich może oczekiwać Klient i Pracownik stacji kontroli pojazdów. Na podstawie opracowania wyłonione zostały elementy systemu, jakie należało zaprojektować oraz zaprogramować.

3.1. Wymagania funkcjonalne

W tym podrozdziale zdefiniowane zostały wymagania funkcjonalne. Zakładamy podział ról użytkowników na następujących aktorów:

- Użytkownik niezalogowany,
- Klient,
- Pracownik,
- Administrator.

Wymagania funkcjonalne dla Użytkownika niezalogowanego

- możliwość zarejestrowania nowego konta,
- możliwość zalogowanie do istniejącego konta,
- możliwość przeglądania zawartości serwisu internetowego, która jest jednakowa dla wszystkich użytkowników.

Wymagania funkcjonalne dla Klienta

- wszystkie wymagania funkcjonalne użytkownika niezalogowanego,
- możliwość zarezerwowania terminu usługi za pomocą formularza:
    - wybór posiadanego pojazdu z listy,
    - wybór dostępnego terminu wykonania badania technicznego,
    - zatwierdzenie rezerwacji.
- możliwość przeglądania dokonanych rezerwacji,
- możliwość anulowania rezerwacji,
- możliwość przeglądania historii wykonanych usług,
- możliwość edycji konta,
- możliwość zmiany hasła do konta,
- możliwość dodania informacji o swoim pojeździe,
- możliwość wylogowania się z konta,

- możliwość usunięcia konta.

Wymagania funkcjonalne dla Pracownika

- wszystkie wymagania funkcjonalne Klienta z wyłączeniem usuwania konta,
- możliwość przeglądania kalendarza pracy,
- możliwość wprowadzania informacji na temat wykonanej usługi:
    - możliwość sporządzenia raportu o przebiegu usługi,
    - możliwość wprowadzenia informacji na temat wyniku przeprowadzonego badania,
- możliwość podglądu szczegółów rezerwacji.

Wymagania funkcjonalne dla Administratora

- wszystkie wymagania funkcjonalne Pracownika,
- możliwość zarządzania kontami klientów:
    - przeglądanie informacji i danych klientów,
    - edytowanie danych klientów,
    - usuwanie kont klienckich,
    - zmienianie hasła klienta.
- możliwość zarządzania kontami pracowniczymi:
    - możliwość zakładania kont pracowniczych,
    - możliwość wprowadzania zmian w kontach pracowniczych,
    - możliwość usuwania kont pracowniczych.
- możliwość odwoływania rezerwacji Klientów,
- możliwość aktywacji/dezaktywacji terminów w kalendarzu,
- możliwość zarządzania tygodniem pracy:
    - ustalanie godzin pracy w danym dniu roboczym,
    - ustalanie wolnych dni tygodnia od pracy.
- możliwość ustalania godzin pracy i dni wolnych od pracy,
- możliwość zarządzania stanowiskami do przeprowadzania badań technicznych:
    - możliwość dodawania stanowisk,
    - możliwość edycji nazwy stanowiska,
    - możliwość aktywacji/dezaktywacji stanowiska,
    - możliwość usunięcia stanowiska.

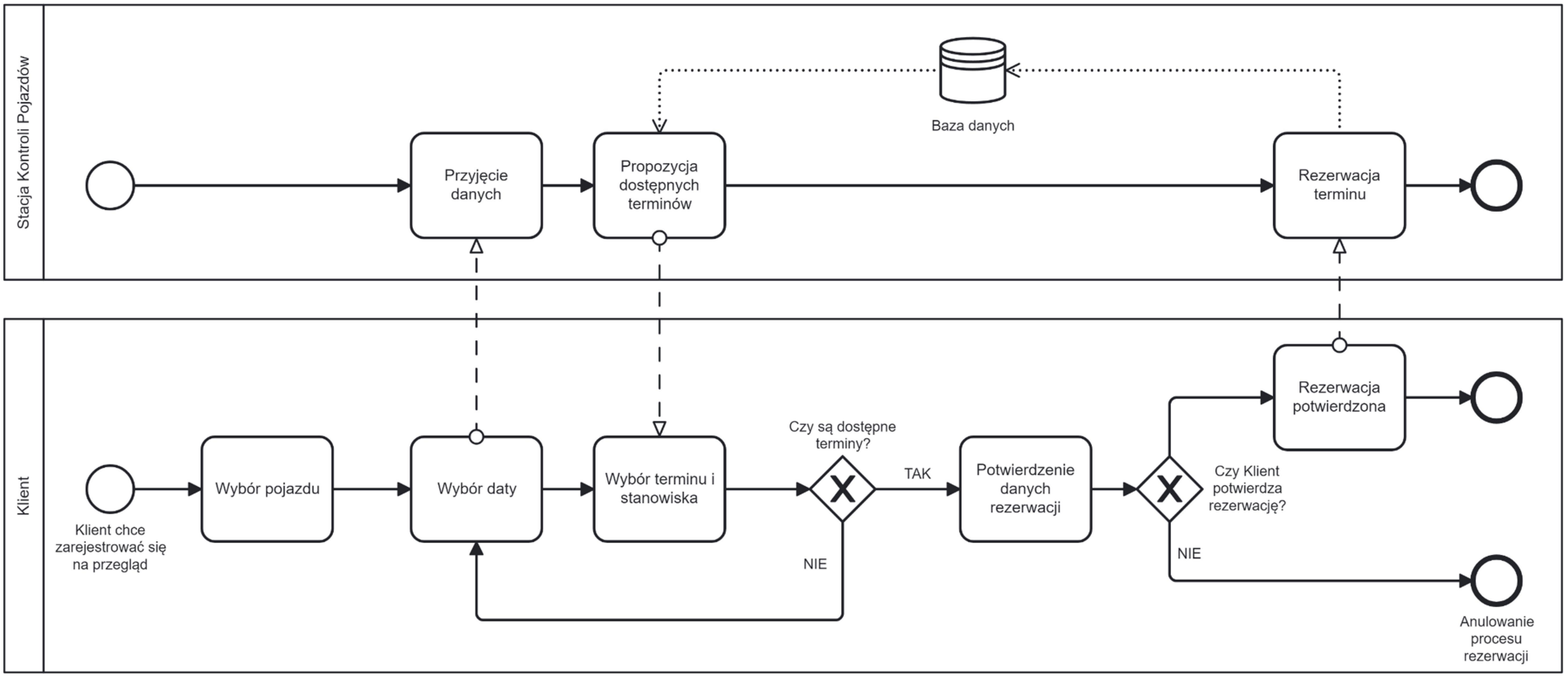
3.2. Wymagania niefunkcjonalne

W tym podrozdziale zdefiniowane zostały wymagania niefunkcjonalne.

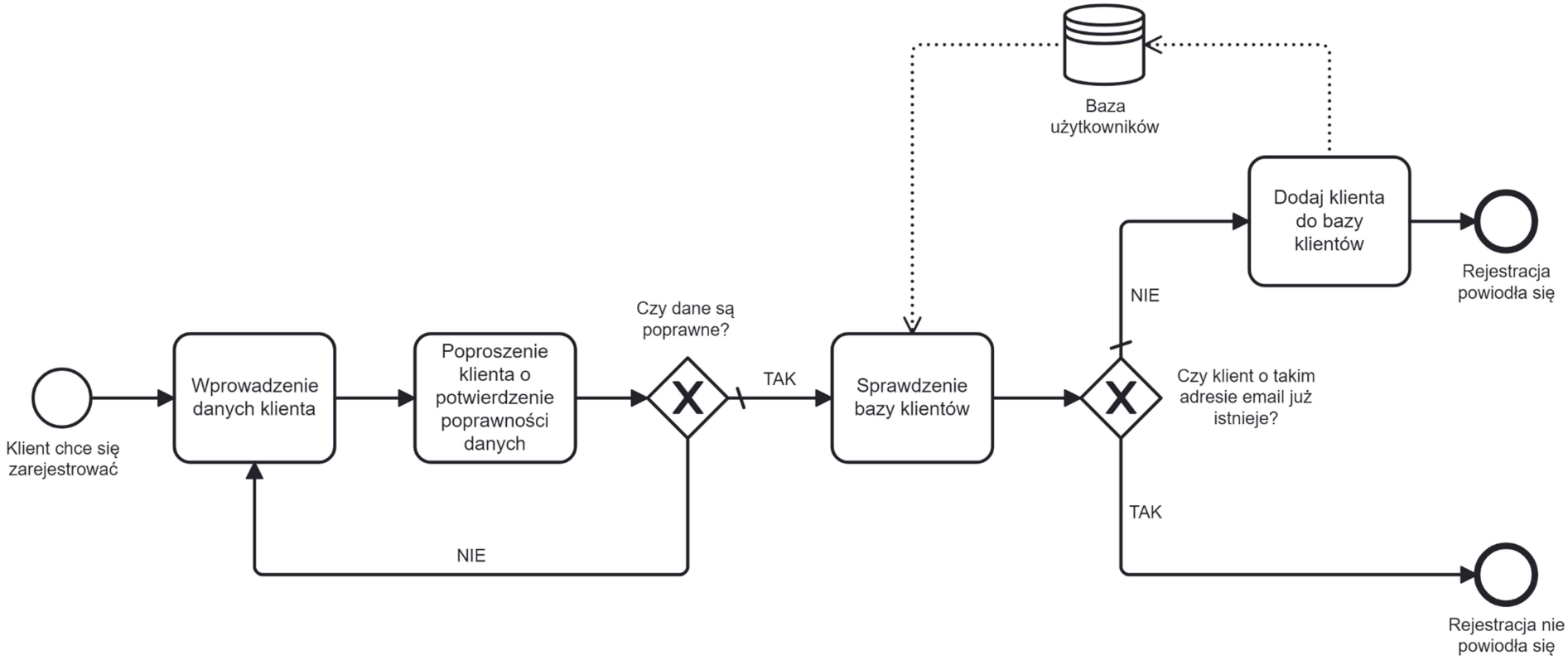
- uwierzytelnienie użytkowników odbywać ma się za pomocą mechanizmu bezstanowych tokenów JWT,
- system ma spełniać założenia skalowalności,
- hasła kont przechowywanych w bazie danych mają być szyfrowane,
- aplikacja dostępowa ma być aplikacją internetową kompatybilną z przeglądarkami internetowymi: Chrome, Firefox, Edge, Opera, Safari,
- interfejs Klienta typu GUI ma być łatwy w obsłudze i intuicyjny. Korzystanie z interfejsu użytkownika powinno być możliwe bez przeszkolenia,
- aplikacja internetowa powinna spełniać założenia responsywności oraz wyświetlać się prawidłowo na urządzeniach typu landscape oraz portrait,
- system informatyczny musi zachowywać zgodność z obowiązującymi w Polsce przepisami RODO oraz innymi opisanymi w Rozdziale 1,
- system informatyczny wykorzystuje relacyjną bazę danych PostgreSQL,
- warstwa serwerowa aplikacji internetowej wykonana za pomocą technologii Spring w języku Java, a warstwa kliencka wykonana za pomocą technologii React w języku JavaScript,
- do uruchomienia aplikacji internetowej na serwerze wymagane są narzędzia Docker Engine oraz Docker Compose.

3.3. Diagramy procesów biznesowych

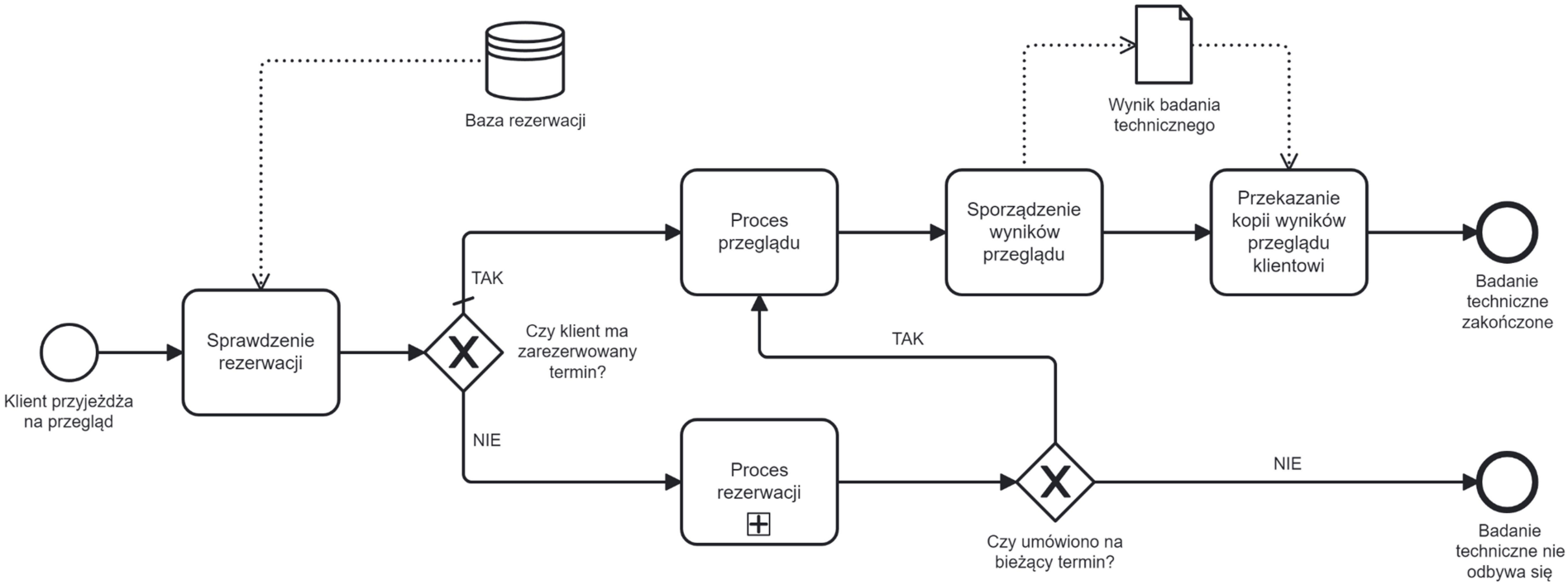
Diagramy standardu BPMN (Buisness Process Modeling Notation) służą do graficznego opisu procesów biznesowych, jakie mają miejsce w firmie. Jest to istotny element dziedziny Inżynierii Oprogramowania. Modelowanie procesów biznesowych pozwala na szybką możliwość zrozumienia metodyki działania firmy przez osobę znającą standard bez konieczności osobistej bezpośredniej obserwacji procesów biznesowych. Na diagramach widoczne są przepływy informacji, role aktorów, miejsca powstawania oraz przechowywania dokumentów oraz różnice semantyczne między nimi i wiele więcej. Diagramy sporządzono zgodnie z wiedzą zaczerpniętą z literatury [5]. Na rysunku 3.1 przedstawiono diagram procesu biznesowego rezerwacji usługi. Rysunek 3.2 pokazuje diagram procesu biznesowego rejestracji konta w systemie. Na rysunku 3.3 widnieje schemat procesu biznesowego jakim jest przeprowadzanie badania technicznego.



Rysunek 3.1. Diagram procesu biznesowego rezerwacji terminu usługi



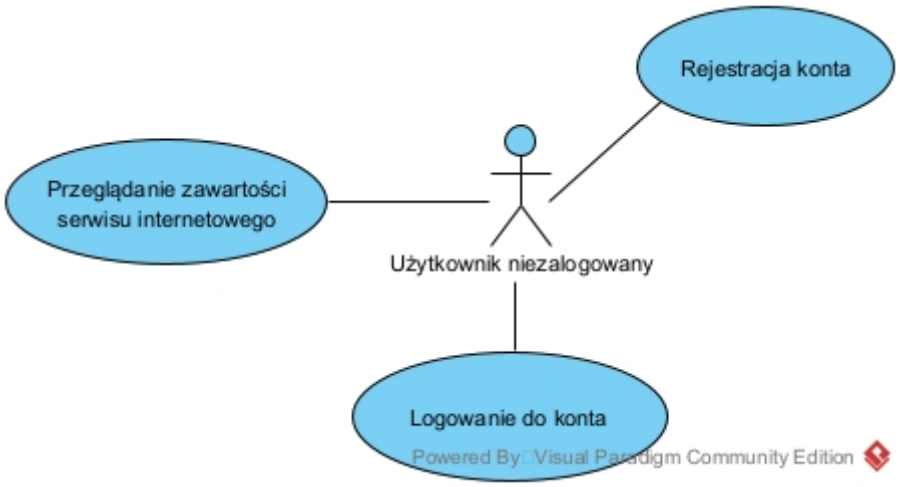
Rys. 3.2. Diagram procesu biznesowego rejestracji konta w systemie



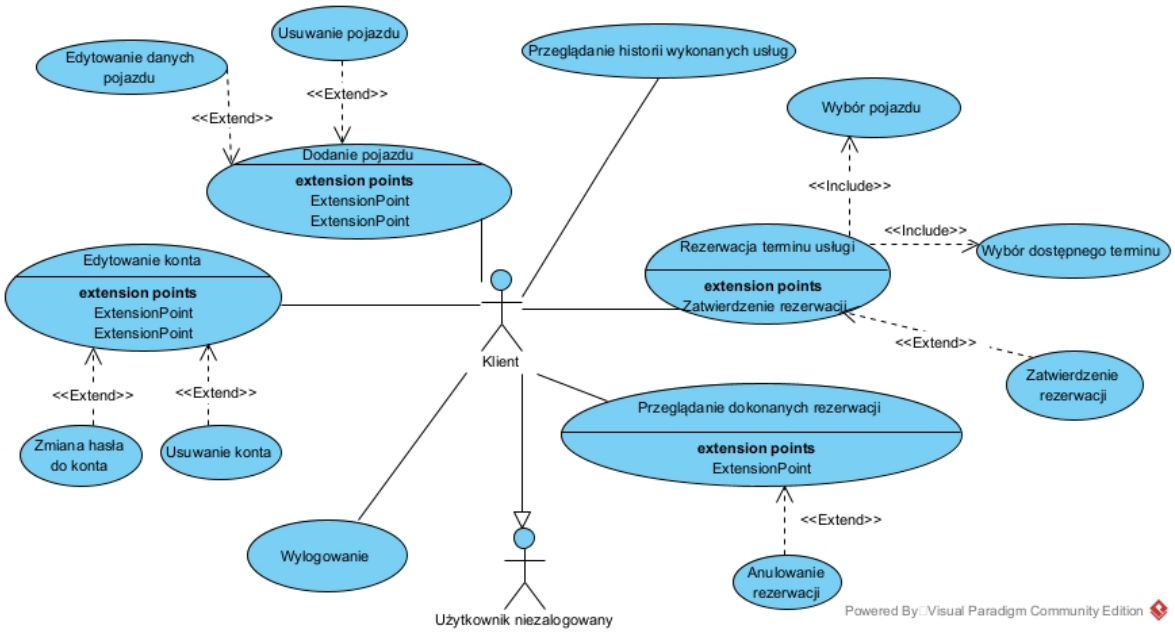
Rys. 3.3. Diagram procesu biznesowego - Badanie techniczne

3.4. Diagramy przypadków użycia (DPU)

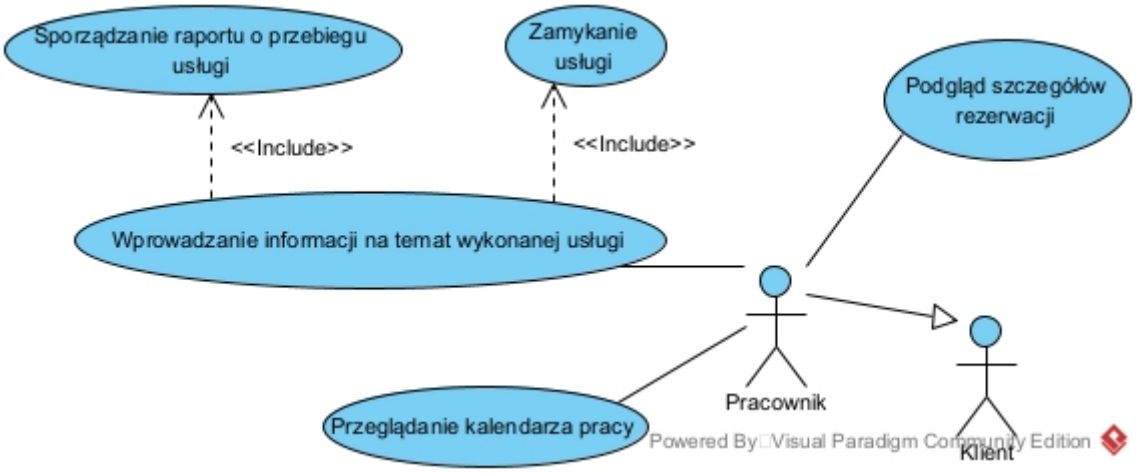
Diagramy przypadków użycia to graficzne reprezentacje wymagań funkcjonalnych systemu. Ukazują one interakcje pomiędzy aktorami, a zdarzeniami w systemie. Diagram przypadku użycia na rysunku 3.4 przedstawia funkcjonalności jakimi dysponuje Użytkownik niezalogowany. Następnie rysunki 3.5, 3.6 oraz 3.7 przedstawiają bardziej złożone diagramy odpowiednio dla Klienta, Pracownika oraz Administratora, wraz z obowiązkowymi i opcjonalnymi realizacjami przypadków użycia. Informacje o przeznaczeniu oraz sposobie konstruowania diagramów przypadków użycia zostały pozyskane z literatury [7].



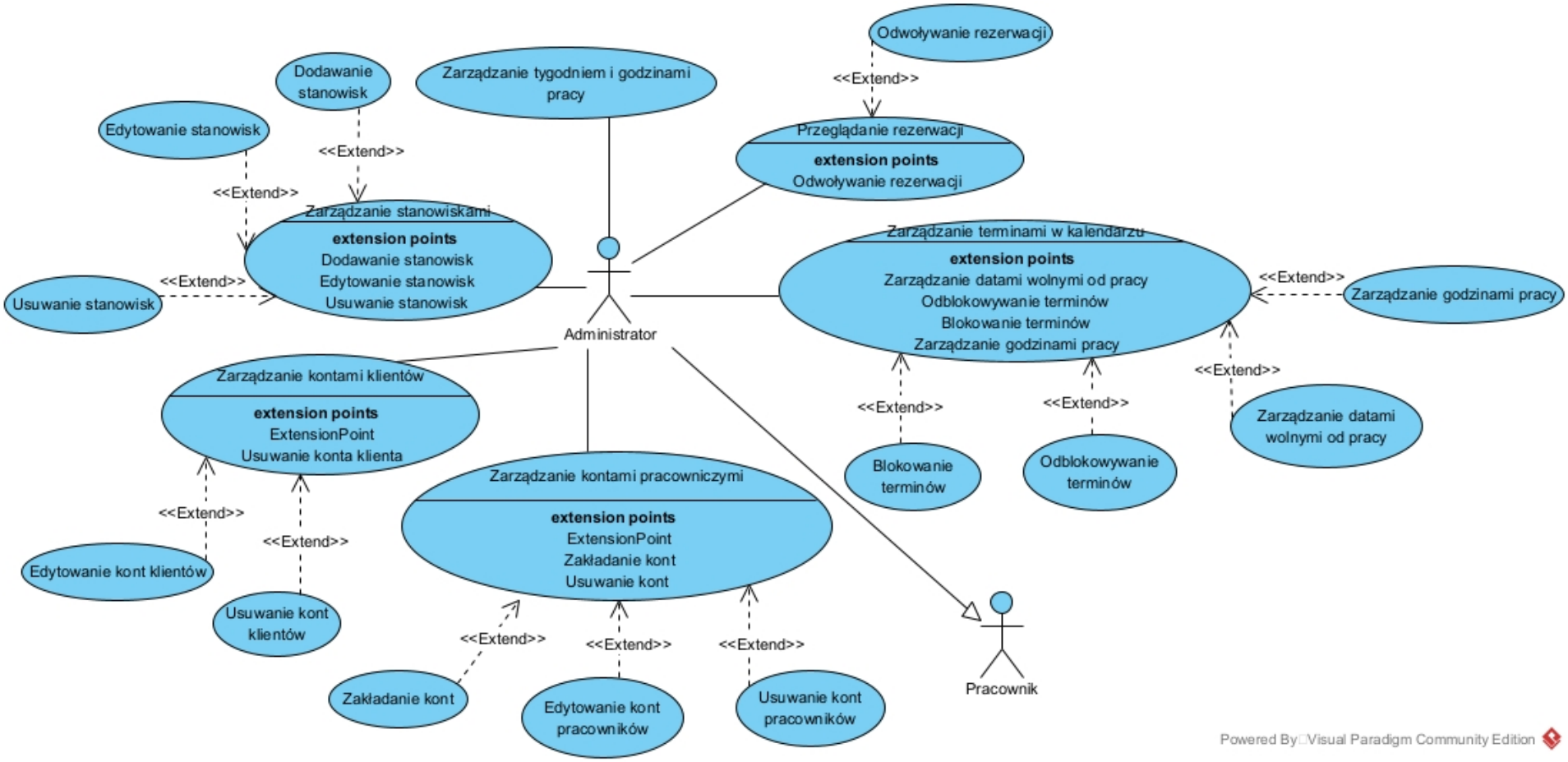
Rysunek 3.4. Diagram DPU dla Użytkownika niezalogowanego



Rysunek 3.5. Diagram DPU dla Klienta



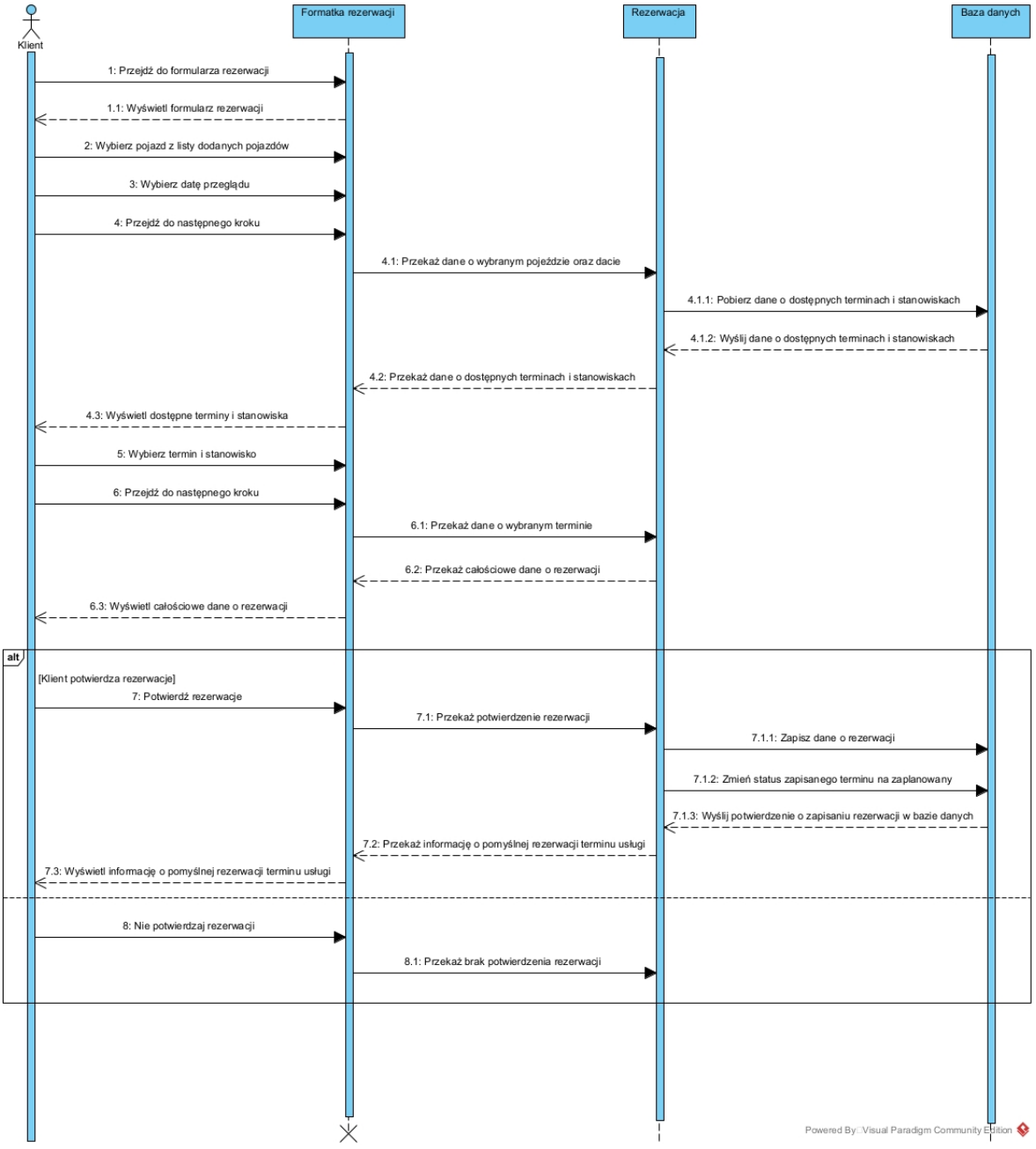
Rysunek 3.6. Diagram DPU dla Pracownika



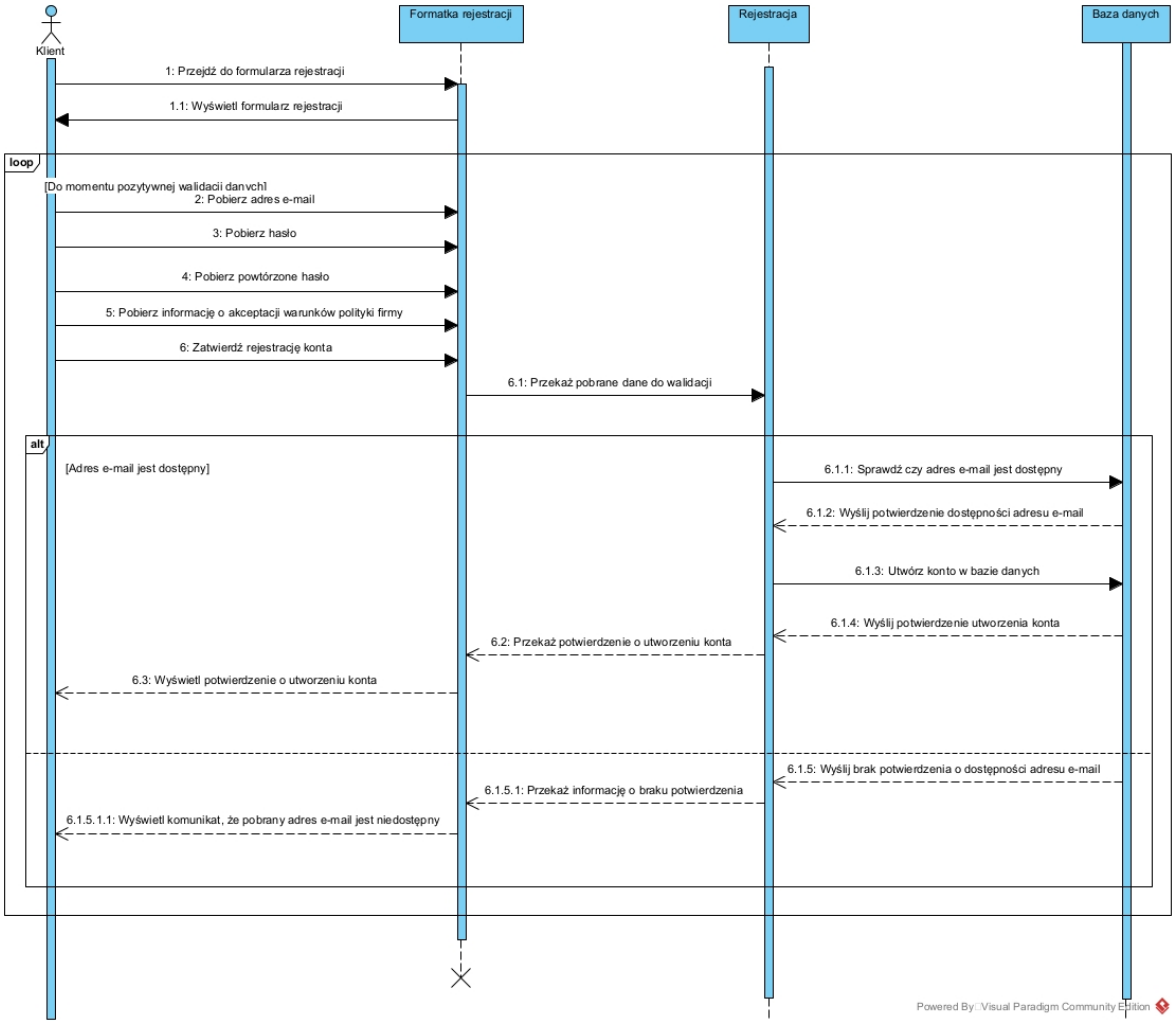
Rysunek 3.7. Diagram DPU dla Administratora

3.5. Diagramy sekwencji

Diagramy sekwencji, są częścią języka UML. Służą do modelowania dynamicznych aspektów systemu. Obrazują to, jak obiekty systemu wymieniają komunikaty w trakcie realizacji określonej funkcjonalności. Dzięki nim możliwe jest zobrazowanie przepływu informacji między aktorem a systemem, co pozwala na lepsze zrozumienie zachowania systemu w konkretnych sytuacjach. Na rysunkach 3.8 i 3.9 przedstawione zostały diagramy sekwencji dla obsługi rezerwacji terminu usługi oraz rejestracji konta klienta. Wiedza o diagramach sekwencji standardu UML jest zgodna z literaturą [7].



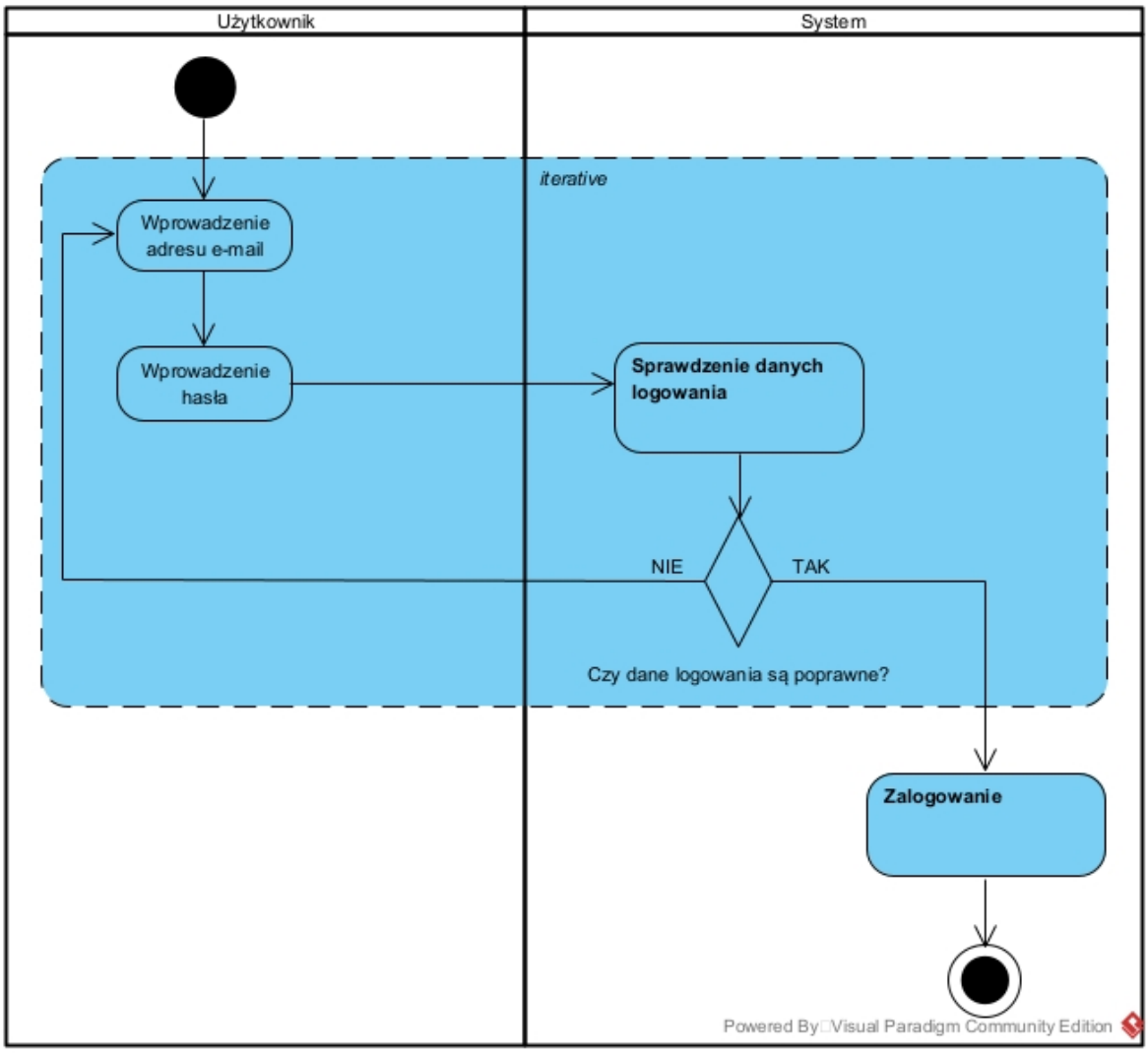
Rysunek 3.8. Diagram sekwencji obsługi rezerwacji terminu badania technicznego



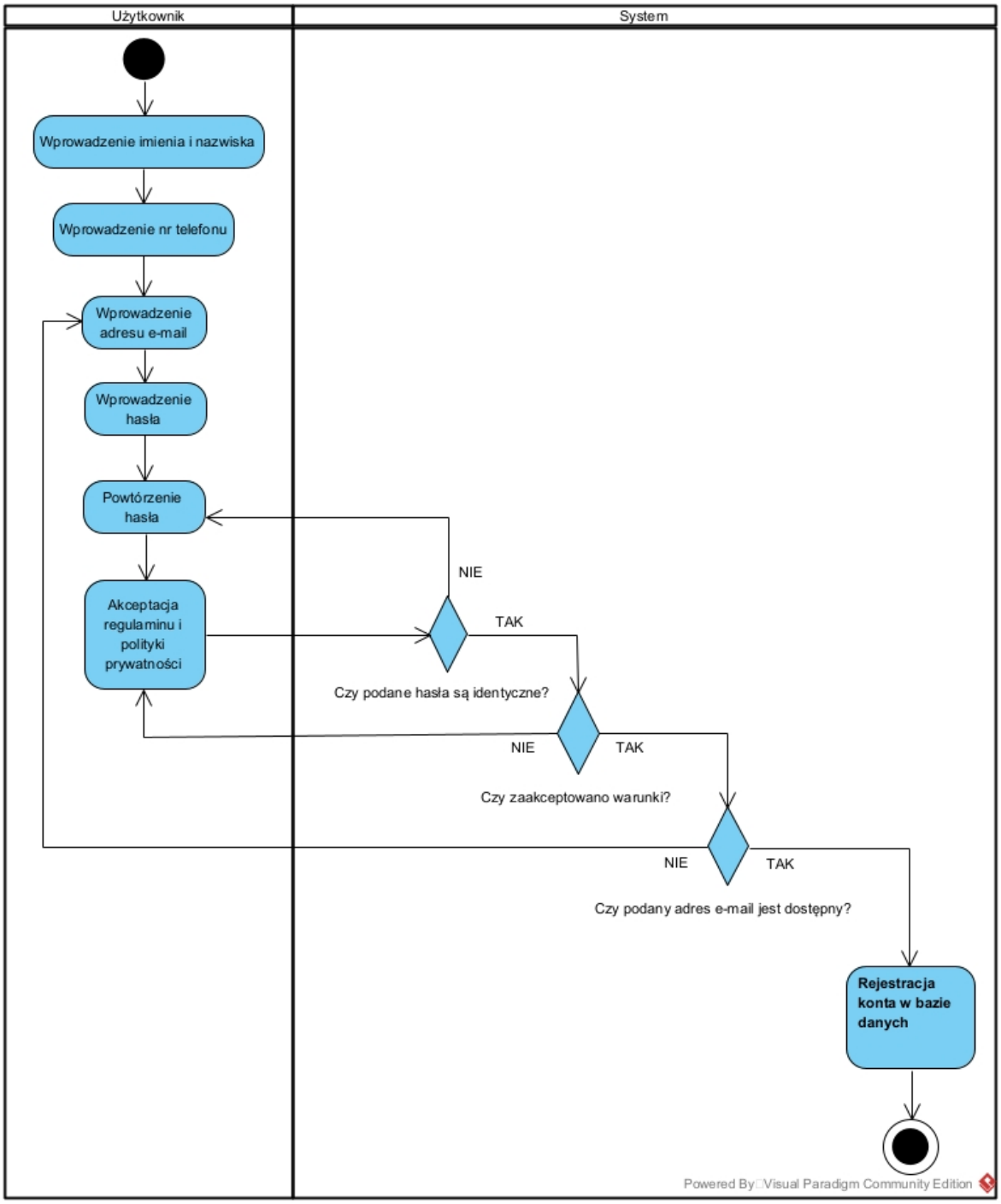
Rysunek 3.9. Diagram sekwencji obsługi rejestracji konta klienta

3.6. Diagramy aktywności

Diagramy aktywności (czynności) są kolejną częścią języka UML. Służą one do obrazowania wewnętrznych procesów projektowanego systemu. Procesy te są dzielone na aktywności (czynności), pomiędzy którymi następuje przepływ sterowania oraz obiektów. Przepływy określają kolejność wykonania aktywności. Rysunek 3.10 prezentuje diagram aktywności logowania do systemu. Diagram aktywności rejestracji nowego konta w systemie prezentuje rysunek 3.11. Fragment standardu UML mówiący o regułach tworzenia diagramów aktywności przedstawiony jest w literaturze [7].



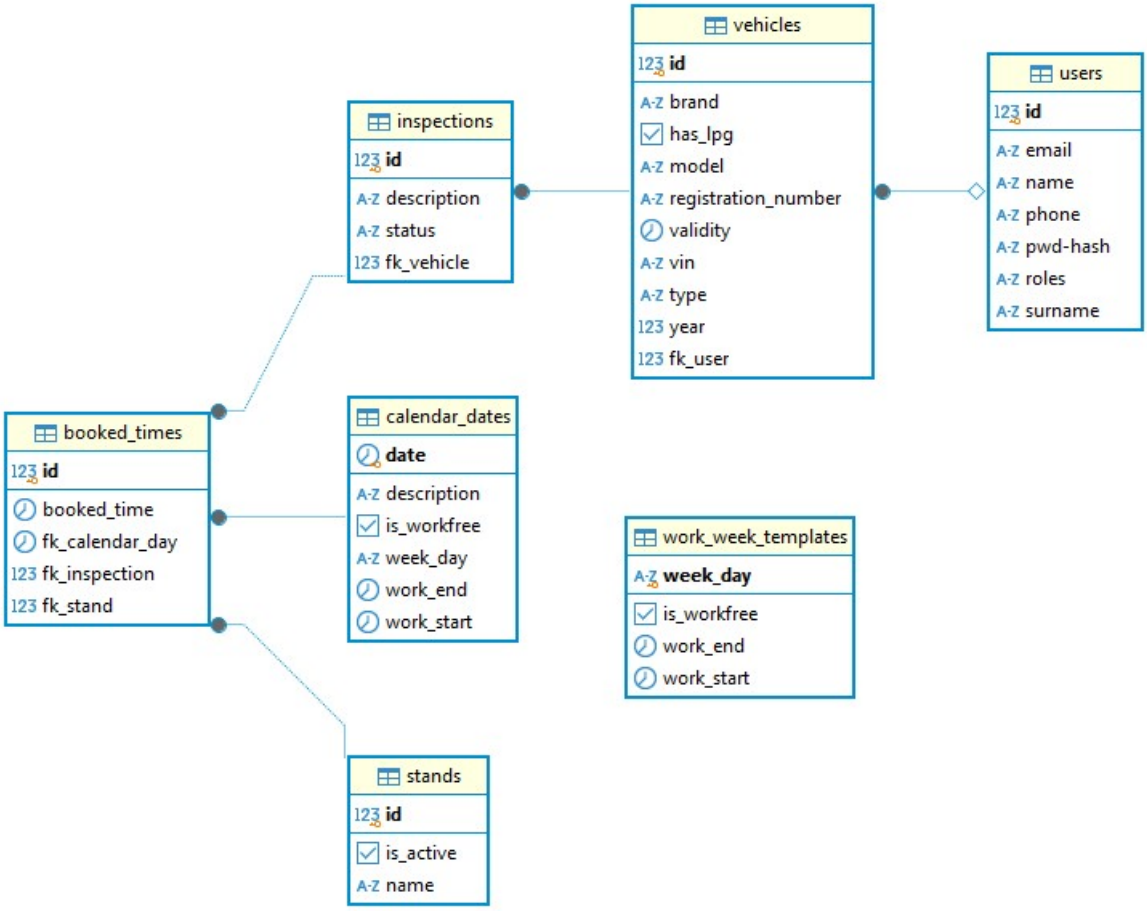
Rysunek 3.10. Diagram aktywności logowania do systemu



Rysunek 3.11. Diagram aktywności rejestracji nowego konta klienta w systemie

3.7. Diagram ERD (Entity Relationship Diagram) bazy danych

System został oparty na relacyjnej bazie danych przedstawionej poniżej na rysunku 3.12. Baza danych została zaprojektowana w możliwie jak najprostszy i najefektywniejszy sposób, aby ułatwić komunikacje poszczególnych encji za pomocą relacji, przy zachowaniu wszystkich wymagań funkcjonalnych przewidzianych dla tego systemu. System bazodanowy projektowanego systemu wykonano w technologii PostgreSQL. Jest to relacyjna baza danych oparta na platformie open-source.



Rysunek 3.12. Diagram ERD wygenerowany za pomocą narzędzia DBeaver na podstawie encji utworzonych przez Hibernate

Na rysunku 3.12 nie widać połączenia encji „work_week_templates” z pozostałą częścią diagramu. Dzieje się tak dlatego, ponieważ ta encja nie jest połączona z żadną inną encją w diagramie bezpośrednią relacją bazodanową. Nie można jednak na tej podstawie postawić tezy, iż encja ta jest niepotrzebna w systemie, gdyż to połączenie jest realizowane w wyższych warstwach serwera. Poprawną tezą jest sformułowanie: połączenie pomiędzy encją „work_week_templates” a encją „calendar_dates” realizowane jest programowo

w warstwie usług serwera aplikacji. Przedstawienie takiego wysokopoziomowego połączenia za pomocą relacji byłoby w oczywisty sposób niezgodne ze standardami sporządzania diagramów ERD relacyjnych baz danych, gdyż powiązanie to jest częścią logiki biznesowej aplikacji, a nie relacją bazodanową.

3.8. Projekt interfejsu

Projekt interfejsu jest zestawem widoków, które widzi użytkownik podczas użytkowania aplikacji. Interfejs był tworzony z zamysłem o maksymalnej prostocie i intuicyjności użytkowania. Założeniem projektowanego interfejsu jest dostosowanie aplikacji do jak najszerszej grupy użytkowników. Motoryzacja jest branżą zrzeszającą zarówno osoby młodsze jak i starsze. W celu utworzenia komfortowego i przejrzystego interfejsu wykorzystana została specjalistyczna literatura [11] z dziedziny wiedzy o pozycjonowaniu elementów w ten sposób, aby uczynić go przyjaznym użytkownikowi. Należało zatem zbalansować złożoność pomiędzy młodszymi, których umiejętności informatyczne często są wysokie, a starszymi użytkownikami, którzy czasem mogą mieć większe problemy z użytkowaniem aplikacji. Dlatego zapadła decyzja o wykorzystaniu wzorca projektowego MPA (Multi-Page Application), gdzie widoki ograniczają się do prostych, wyizolowanych składowych interfejsu komunikacji użytkownika z systemem informatycznym. Nie oznacza to jednak nie wykorzystania założeń aplikacji SPA (Single-Page Application), m.in. nie ma konieczności pełnego ładowania zawartości każdej podstrony, następuje jedynie wymiana zmieniającej się zawartości. Takie możliwości i wiele innych oferują nowoczesne frameworki. W niniejszej pracy wykorzystany jest szablon React opierający się na języku JavaScript.

Projekt interfejsu obejmuje następujące elementy:

1. Stronę tytułową – na rysunkach 3.13 i 3.14 przedstawiona jest podstrona początkowa widoczna po odwiedzeniu domeny. Są tam umieszczone najważniejsze informacje biznesowe, odnośniki kierujące do informacji ze świata motoryzacji, a także grafiki tematyczne.

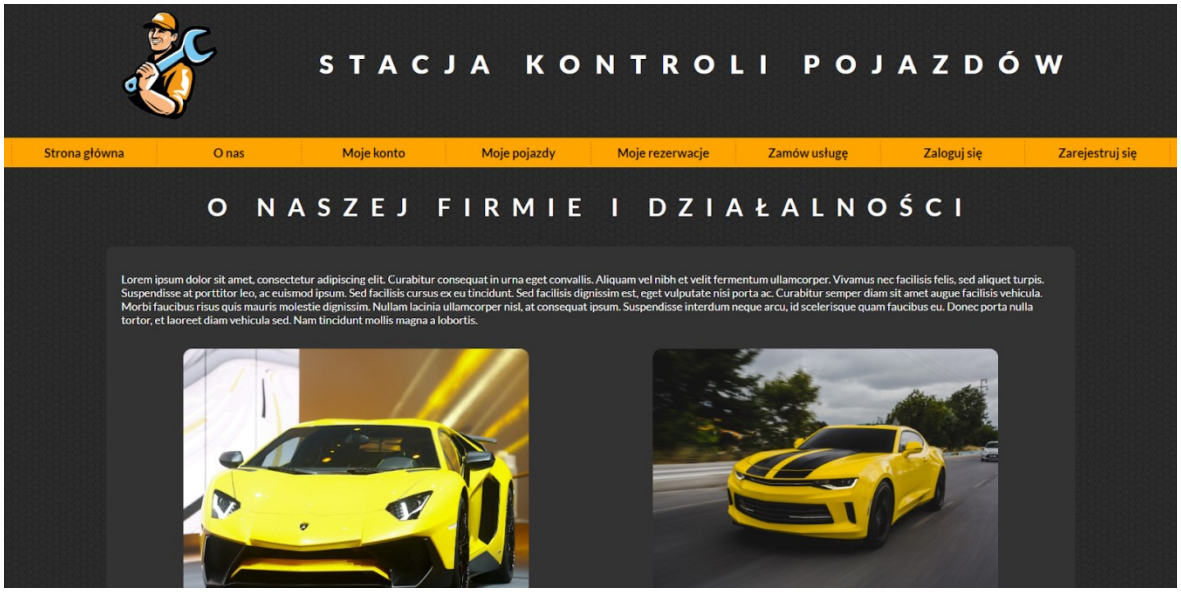


Rysunek 3.13. Makieta strony głównej



Rysunek 3.14. Makieta strony głównej (ciąg dalszy)

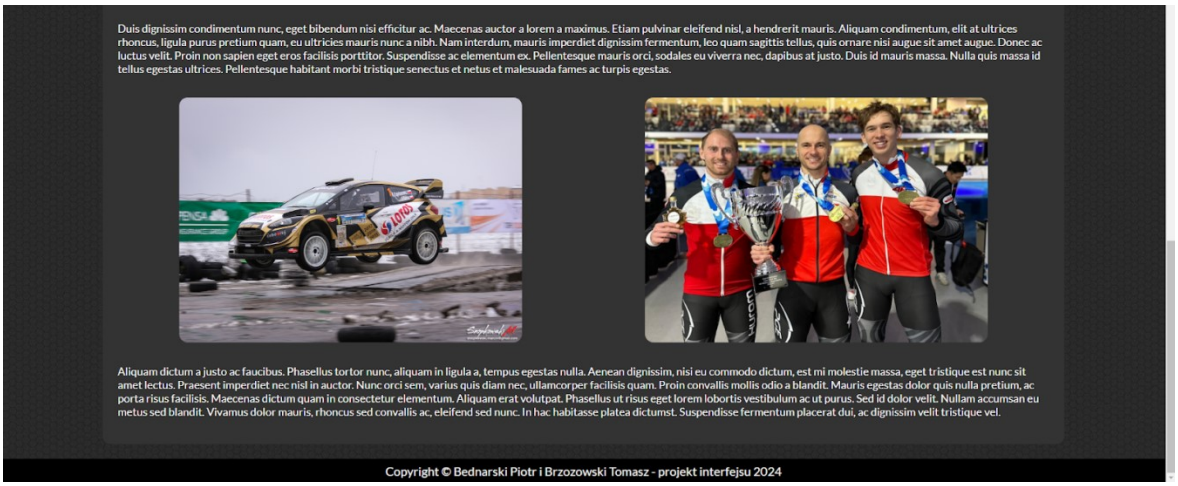
2. Podstronę “O nas” – na rysunkach 3.15, 3.16 i 3.17 przedstawiony jest widok dotyczy zalogowanych i niezalogowanych użytkowników. Znajdują się tam informacje o prowadzonej działalności gospodarczej, historia firmy oraz inne tematyczne informacje.



Rysunek 3.15. Makieta podstrony “O nas” (część 1/3)



Rysunek 3.16. Makieta podstrony “O nas” (część 2/3)



Rysunek 3.17. Makieta podstrony “O nas” (część 3/3)

3. Zakładkę “Moje konto” – na rysunku 3.18 przedstawiony jest widok umożliwiający zarządzanie danymi zgromadzonymi na koncie użytkownika. Dotyczy wyłącznie zalogowanych Klientów.



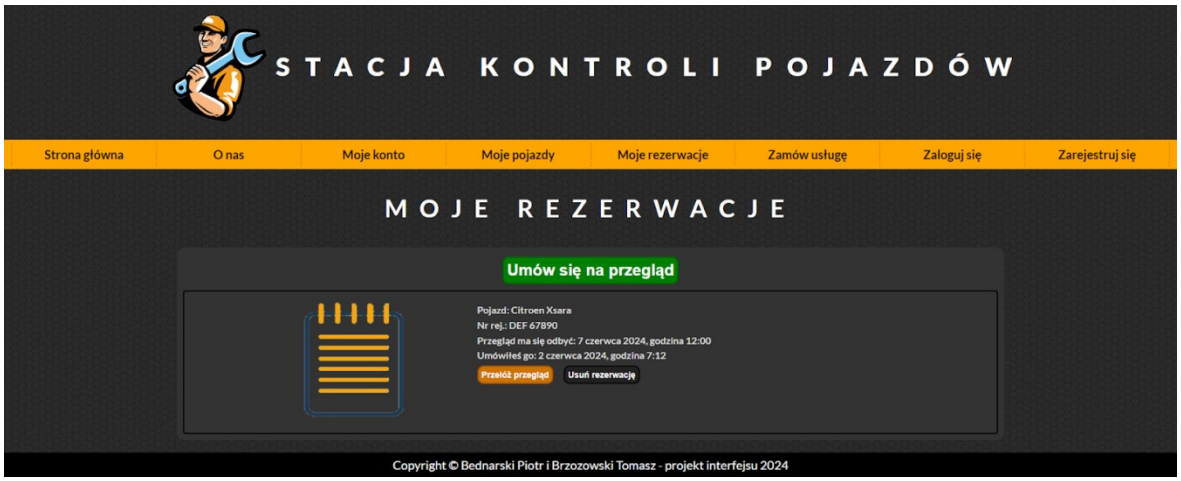
Rysunek 3.18. Makieta podstrony “Moje konto”

4. Zakładkę “Moje pojazdy” – na rysunku 3.19 przedstawiony jest widok umożliwiający zarządzanie danymi o pojazdach przypisanych do Konta użytkownika. Za pomocą tego widoku jest możliwe dodawanie, usuwanie oraz edycja danych o pojazdach. Dotyczy wyłącznie Klientów zalogowanych.



Rysunek 3.19. Makieta podstrony “Moje pojazdy”

5. Zakładkę “Moje rezerwacje” – na rysunku 3.20 przedstawiony jest widok umożliwiający tworzenie, przeglądanie, edycję oraz anulowanie rezerwacji przypisanych do tego Konta użytkownika. Dotyczy jedynie użytkowników zalogowanych.



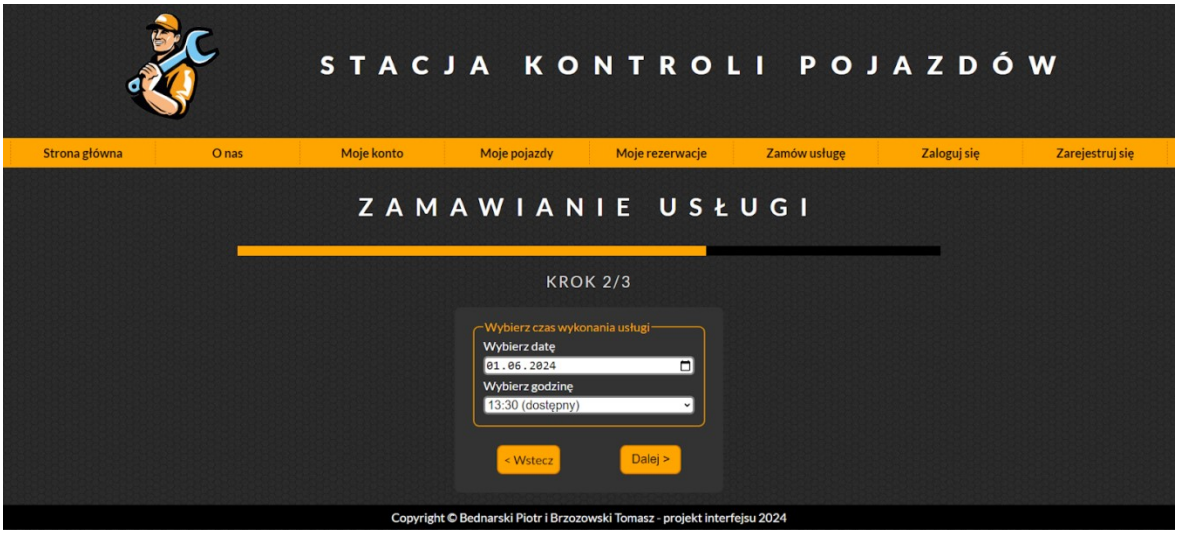
Rysunek 3.20. Makieta podstrony “Moje rezerwacje”

6. Zakładkę “Zamów usługę” – na rysunkach 3.21, 3.22 i 3.23 przedstawiony jest widok zawierający formularz służący do rezerwacji wizyty. Rozbity on został na trzy osobne kroki semantyczne po to, aby użytkownik nie miał wrażenia przytłoczenia przez dużą

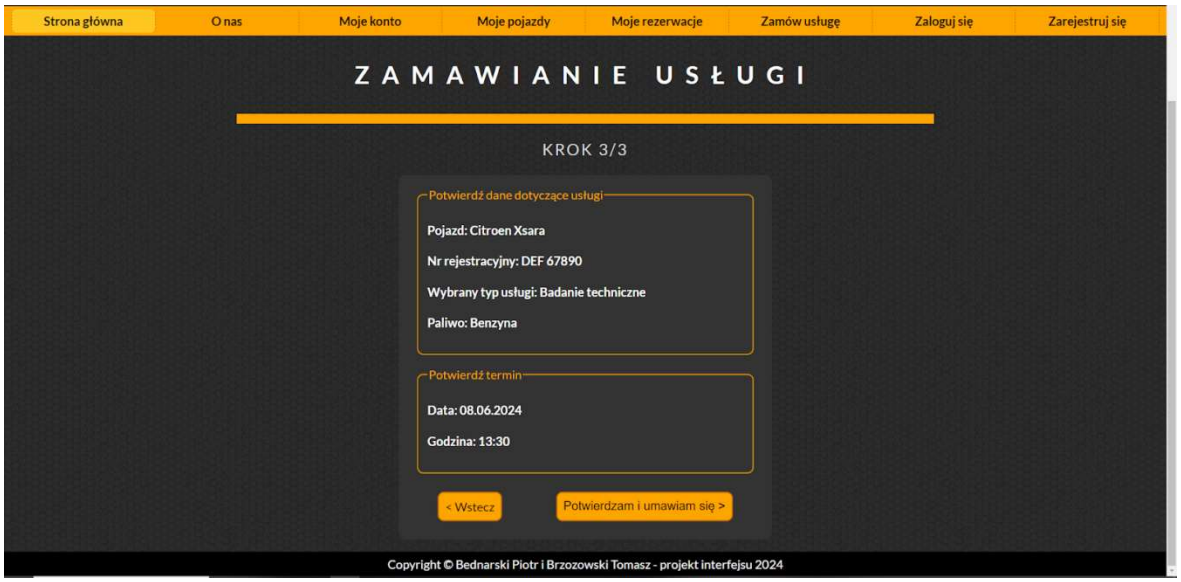
ilość koniecznych danych do wprowadzenia. Dotyczy tylko użytkowników zalogowanych.



Rysunek 3.21. Makieta podstrony “Zamawianie usługi” (krok 1 z 3)



Rysunek 3.22. Makieta podstrony “Zamawianie usługi” (krok 2 z 3)



Rysunek 3.23. Makieta podstrony “Zamawianie usługi” (krok 3 z 3)

7. Zakładkę “Logowanie” – na rysunku 3.24 przedstawiony jest widok zawierający formularz logowania. Za pomocą tego formularza użytkownik autoryzuje swój dostęp do konta. Widok dotyczy użytkowników niezalogowanych.



Rysunek 3.24. Makieta formularza logowania do systemu

8. Zakładkę “Rejestracja” – na rysunku 3.25 przedstawiony jest widok służący do tworzenia nowego konta użytkownika. Dotyczy użytkowników niezalogowanych.



Rysunek 3.25. Makieta formularza rejestracji konta

4. Implementacja systemu

W niniejszym rozdziale przedstawiony został proces implementacji systemu informatycznego będącego tematem pracy. Omówiony został wybór stosu technologicznego wraz z opisem wybranych technologii oraz przedstawiony proces implementacji poszczególnych funkcjonalności zarówno po stronie klienta jak i serwera.

4.1. Użyte technologie

PostgreSQL

Jako technologię bazodanową wybrana została relacyjna bazę danych PostgreSQL. Ta decyzja została podjęta ze względu na fakt, iż bazy danych PostgreSQL są stabilne, chętnie wykorzystywane przez firmy zajmujące się produkcją oprogramowania a także ze względu na to, że realizacja połączenia pomiędzy tą bazą serwerem w Spring Framework poprzez JPA (Java Persistence API) jest bardzo dobrze udokumentowane.

Spring Boot

Do implementacji warstwy serwera w postaci REST API wybrany został Spring Framework, gdyż jest to niezwykle popularny szkielet umożliwiający maksymalne uproszczenie wykonywania takich operacji jak łączenie się z bazą danych poprzez wysokopoziomowe API jakim jest Hibernate, podział oprogramowania serwerowego na warstwy logiczne: trwałości, usług i prezentacji, wystawianie endpointów w architekturze REST za pomocą mechanizmu kontrolerów czy realizację mechanizmu autoryzacji i autentykacji użytkowników za pomocą Spring Security. Ważnym aspektem w Spring Framework jest możliwość korzystania z mechanizmu Dependency Injection. Dodatkową zaletą korzystania z tego frameworka jest możliwość konfiguracji projektu w bardzo szerokim zakresie oraz fakt istnienia bardzo rozbudowanej dokumentacji na temat Spring Framework. Skorzystaliśmy również z warstwy Spring Boot położonej na warstwie Spring Framework, która m.in. dostarcza autokonfigurację projektu oraz kilka przydatnych, wysokopoziomowych adnotacji języka Java. Użycie Spring Boot (Spring Framework) pozwala na nie skupianiu się na wyżej wymienionych aspektach, a jedynie na implementacji logiki biznesowej aplikacji.

Do implementacji serwera użyto Spring Boot w wersji 3.4.0 oraz języka Java na poziomie JDK 21.

Wykorzystano także Project Lombok w wersji 1.18.32, czyli narzędzie deweloperskie języka Java pozwalające na znaczne ograniczenie tzw. boiler-plate code przy użyciu wysokopoziomowych adnotacji, które Lombok dostarcza.

Do realizacji autoryzacji oraz autentykacji użytkowników z podziałem na role wykorzystano pakiet Spring Security w wersji 3.4.0.

Logika połączenia z bazą danych nawiązywane jest za pomocą technologii Hibernate z pakietu Spring Data JPA w wersji 3.3.4.

Mapowanie mniej skomplikowanych obiektów encji na klasy typu dto wykonane jest przy użyciu pakietu Model Mapper w wersji 3.0.0.

React

Warstwa klienta została zaimplementowana przy użyciu popularnej biblioteki React w języku Java Script, która umożliwia budowę interfejsów użytkownika. Biblioteka wspiera tworzenie oraz zarządzanie rozbudowanymi aplikacjami. Dzieje się tak, poprzez zastosowanie podziału całej logiki aplikacji na komponenty. Są nimi odizolowane od siebie fragmenty kodu, które w kontekście całej aplikacji są niewielkie, lecz złożone w jedną całość tworzą pełny interfejs użytkownika. Zarządzanie aplikacją w React jest przyjazne z punktu widzenia programisty, gdyż wykorzystany został w bibliotece wirtualny DOM (Document Object Model), czyli kopia dokumentu HTML przechowywana w pamięci aplikacji. Wirtualny DOM wykrywa różnice z jego rzeczywistym odzwierciedleniem i natychmiastowo go aktualizuje. Takie rozwiązanie sprawia, że tworzenie aplikacji jest dynamiczne. Dodatkowo biblioteka umożliwia nam wykorzystanie języka JSX (JavaScript XML), umożliwiającego aplikowanie kodu HTML w komponencie, co upraszcza programowanie aplikacji.

Docker

W celu uniknięcia skomplikowanego procesu uruchamiania systemu, wykorzystana została technologia konteneryzacji Docker. Wykorzystanie mechanizmu konteneryzacji umożliwia rozdzielenie serwisu na mikrousługi. W połączeniu z narzędziem Docker Compose, pozwalającym na łączenie wielu kontenerów Docker w spójną całość możliwe było zaimplementowanie struktury połączeń i zależności między poszczególnymi kontenerami. Dzięki wykorzystaniu tych technologii, do uruchomienie tworzonego systemu na stacji

roboczej, niezbędne jest zainstalowanie jedynie Docker Engine. Ponadto, konteneryzacja umożliwia w bardzo prosty sposób uruchomienie systemu w chmurze obliczeniowej dzięki spełnieniu założeń podejścia IaC – Infrastructure as Code.

Git

Podczas procesu implementacji opisywanego systemu, wykorzystany został rozproszony system kontroli wersji Git. System ten umożliwia śledzenie zmian w projekcie programistycznym, a użycie zdalnego repozytorium GitHub pozwala zespołowi programistów jednoczesną pracę nad aktualną wersją projektu.

4.2. Warstwa serwerowa

Wielowarstwowość serwera

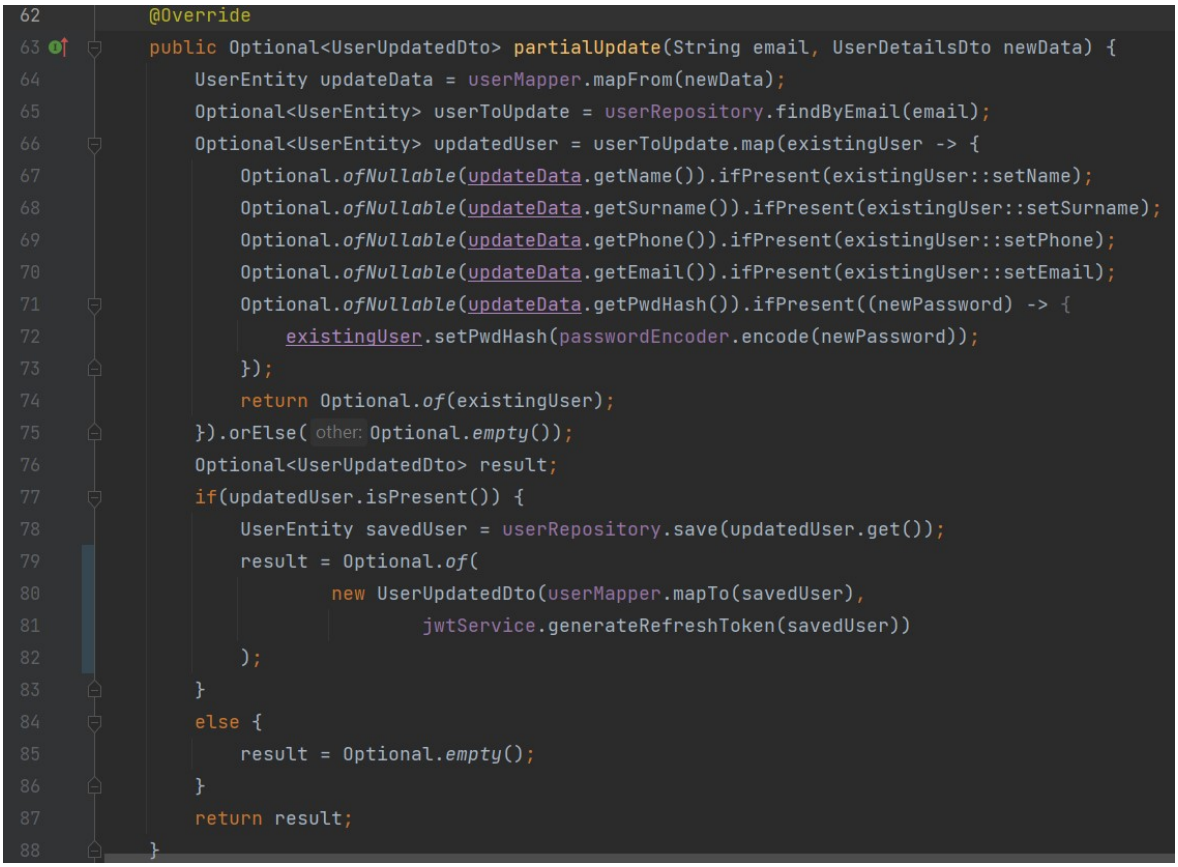
Przy podziale aplikacji serwerowej możemy wyłonić trzy główne warstwy:

- warstwa prezentacji (presentation layer),
- warstwa usług (service layer),
- warstwa trwałości (persistence layer).

Warstwa prezentacji zawiera klasy kontrolerów. Każda z nich wystawia endpoint lub grupę endpointów realizujących określoną funkcjonalność. Zbiór klas kontrolerów wystawia API w architekturze REST (adnotacja @RestController). Wystawienie takiego API spowodowane jest koniecznością umożliwienia dostępu do funkcjonalności serwera warstwie klienckiej. Warto zaznaczyć, iż założeniem tej warstwy jest nie umieszczanie w niej logiki biznesowej aplikacji, a jedynie definicja odpowiednich reakcji serwera na różne, przewidywanie zachowania warstwy klienckiej poprzez generowanie odpowiedzi o odpowiednich kodach protokołu HTTP (użyta klasa generyczna ResponseEntity<>). Zadaniem tej warstwy jest także wywoływanie odpowiedniego fragmentu logiki biznesowej aplikacji (kontrolery posiadają wstrzyknięte klasy warstwy service).

Warstwa usług zawiera klasy opatrzone adnotacją @Service. Założeniem tej warstwy jest implementacja logiki biznesowej aplikacji. Na tę logikę składa się m. in. problematyka mapowania obiektów typu Dto (Data Transfer Objects) na obiekty encji Hibernate i odwrotnie (sama logika mapowania wyeksportowana jest do klas mapperów, jednakże to do tej warstwy mappery są wstrzykiwane), implementacja logiki programowanej funkcjonalności, a także komunikacja z warstwą trwałości. Realizacja tych wszystkich

założeń powoduje wyraźnie najwyższy poziom skomplikowania tego typu klas na tle klas z pozostałych warstw aplikacji, co pokazuje rysunek 4.1.



```
@Override
public Optional<UserUpdatedDto> partialUpdate(String email, UserDetailsDto newData) {
    UserEntity updateData = userMapper.mapFrom(newData);
    Optional<UserEntity> userToUpdate = userRepository.findByEmail(email);
    Optional<UserEntity> updatedUser = userToUpdate.map(existingUser -> {
        Optional.ofNullable(updateData.getName()).ifPresent(existingUser::setName);
        Optional.ofNullable(updateData.getSurname()).ifPresent(existingUser::setSurname);
        Optional.ofNullable(updateData.getPhone()).ifPresent(existingUser::setPhone);
        Optional.ofNullable(updateData.getEmail()).ifPresent(existingUser::setEmail);
        Optional.ofNullable(updateData.getPwdHash()).ifPresent((newPassword) -> {
            existingUser.setPwdHash(passwordEncoder.encode(newPassword));
        });
        return Optional.of(existingUser);
    }).orElse( other: Optional.empty());
    Optional<UserUpdatedDto> result;
    if(updatedUser.isPresent()) {
        UserEntity savedUser = userRepository.save(updatedUser.get());
        result = Optional.of(
                new UserUpdatedDto(userMapper.mapTo(savedUser),
                        jwtService.generateRefreshToken(savedUser))
        );
    }
    else {
        result = Optional.empty();
    }
    return result;
}
```

Rysunek 4.1. Implementacja metody wykonującej aktualizację danych konta użytkownika

Warstwa trwałości zawiera interfejsy repozytoriów Hibernate odpowiedzialnych za implementację logiki wykonywania operacji CRUD lub innych, nieco bardziej złożonych operacji na bazie danych. Warto zaznaczyć, że Java Persistence API automatycznie generuje niezbędną implementację takich interfejsów na podstawie interfejsów generycznych, które określone repozytorium rozszerza (np. JpaRepository<>) oraz na podstawie nagłówków deklarujących metody, czasem opatrzonych dodatkową wskazówką w postaci treści zapytania w języku HQL (Hibernate Query Language) przy użyciu adnotacji @Query, co obrazuje rysunek 4.2.

```
2 usages  Piotrek Bednarski
@Repository
public interface BookedTimeRepository extends JpaRepository<BookedTimeEntity,Long> {

    1 usage  Piotrek Bednarski
    List<BookedTimeEntity> findByDay(CalendarDateEntity day);

    1 usage  Piotrek Bednarski
    @Query("SELECT bt FROM BookedTimeEntity bt WHERE bt.day.date = :date AND bt.time = :time AND bt.stand.id = :standId")
    List<BookedTimeEntity> findByDateTimeAndStand(Date date, LocalTime time, Long standId);

}
```

Rysunek 4.2. Przykład Repozytorium Hibernate definiującego dostęp do zarezerwowanych terminów badań technicznych w relacyjnej bazie danych

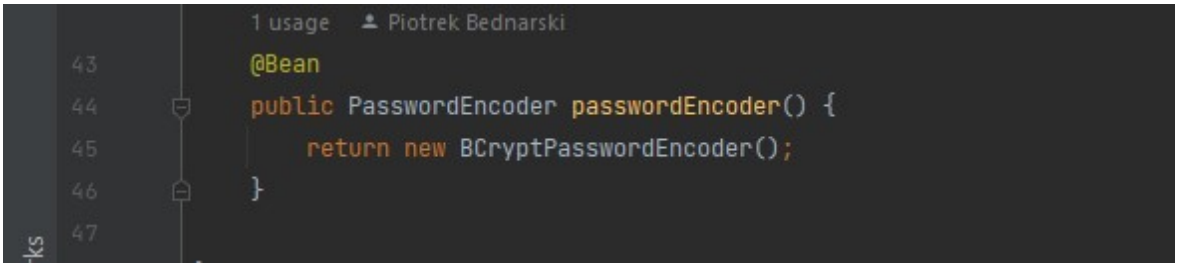
Mechanizm wstrzykiwania zależności

Implementacja warstwy serwerowej została przeprowadzona z wykorzystaniem mechanizmu IoC (Inversion of Control), który w Spring Framework jest dostępny przy użyciu mechaniki DI (Dependency Injection) [14]. Założeniem tego podejścia jest nie tworzenie instancji obiektów klas będących zależnościami w miejscach, gdzie są one potrzebne (np. za pomocą słowa kluczowego „new” w języku Java). Zamiast tego oznaczane są adnotacjami stereotypowymi takimi jak @Component, @Repository, @Service. Adnotacje te w praktyce – oprócz semantycznego przydzielania klasy do odpowiedniej warstwy aplikacji – są informacją dla mechanizmu Spring Framework jakim jest Component Scan uruchamianego przy każdym starcie aplikacji serwerowej, że należy utworzyć instancje tej klasy i umieścić ją w kontekście aplikacji. Kontekst aplikacji jest to kontener, w którym te zależności (dependencies) są umieszczane. Z tego miejsca mogą być automatycznie przez Spring Framework pobierane i wstrzykiwane w odpowiednie miejsca tam, gdzie te instancje są potrzebne. Można zatem w uproszczeniu powiedzieć, że wszystkie klasy oznaczone tymi adnotacjami są @Bean’ami, czyli typami które mają mieć utworzone instancje, a te instancje umieszczone w kontekście aplikacji. Sama adnotacja @Bean również niejednokrotnie pojawia się w kodzie tego projektu. Jest ona stosowana na metodach zwracających instancję obiektu typu takiego jak typ zwracany metody. Przykład użycia tej adnotacji jest przedstawiony na rysunku 4.3. Przedstawiona metoda tworzy konfigurację SecurityFilterChain będącej łańcuchem reguł definiujących politykę SpringSecurity dotycząca mechanizmów bezpieczeństwa aplikacji.



Rysunek 4.3. Przykład Bean'a tworzącego instancję klasy SecurityFilterChain

Wewnątrz metody opatrzonej adnotacją @Bean użycie słowa kluczowego „new" jest już dopuszczalne, co obrazuje rysunek 4.4 przedstawiający konfigurację algorytmu hashującego hasła do kont użytkowników na skróty kryptograficzne tak, aby w bazie danych nie były przechowywane w formie jawnej.



Rysunek 4.4. Algorytm hashujący BCrypt jako algorytm do hashowania haseł

4.3. Warstwa kliencka

Wykorzystanie biblioteki React w procesie implementacji warstwy klienckiej opisywanego systemu informatycznego zostało zmotywowane popularnością wybranej technologii oraz dostępnością dokumentacji na jej temat jak i różnorodnością bibliotek. Jedną z niezbędnych wykorzystanych w projekcie bibliotek jest React Router. Zaprojektowanie tak rozbudowanego interfejsu użytkownika oraz zaprogramowanie tak dużej ilości komponentów było możliwe jedynie za pomocą efektywnego i przemyślanego zarządzania routingiem i nawigacją dostarczonego przez tą bibliotekę [1]. Wykorzystanie komponentów odpowiedzialnych za routing oraz nawigację w aplikacji zostało przedstawione na rysunkach 4.5 oraz 4.6.

```
                { accessToken && role === 'ROLE_ADMIN' &&
                  <Route path='/panel_administratora/rezerwacje' exact element={<Rezerwacje/>}/>
                }
                { !accessToken &&
                  <Route path='/rejestracja' exact element={<Rejestracja onRegister={setAccessToken}/>}/>
                }
                { !accessToken &&
                  <Route path='/logowanie' exact element={<Logowanie onLogin={setAccessToken} />}/>
                }
              </Routes>
            </div>
          </div>
        </BrowserRouter>
    )
}

export default App
```

Rysunek 4.5. Fragment pliku App.jsx

```jsx
const Menu = ({ onLogout }) => {
    const token = localStorage.getItem('access-token')
    const role = localStorage.getItem('role')

    return(
        <header>
            <div>
                <Link to="/">
                    <img src={logo}/>
                </Link>
                <h1>Stacja Kontroli Pojazdów</h1>
            </div>
            <nav>
                <ul>
                    <Link to="/">Strona główna</Link>
                    <Link to="/o_nas">O nas</Link>
                    { token && <Link to="/moje_pojazdy">Moje pojazdy</Link> }
                    { token && <Link to="/moje_rezerwacje">Moje rezerwacje</Link> }
                    { token && <Link to="/zamow_usluge">Zamów usługę</Link> }
                    { token && <Link to="/moje_konto">Moje konto</Link> }
                    { !token && <Link to="/logowanie">Zaloguj się</Link> }
```

Rysunek 4.6. Fragment pliku Menu.jsx

Kolejnym ważnym elementem wykorzystanym w warstwie klienta jest ustandaryzowany proces obsługi wysyłania i odbierania żądań w komponentach. Tworzenie w każdym komponencie kilku metod wysyłających zapytania do warstwy serwerowej jest poprawne lecz generujące duże ilości powtarzającego się kodu. W celu rozwiązania tego problem została skonstruowana metoda przedstawiona poniżej na rysunku 4.7. Można określić najważniejsze argumenty tej metody:

- adres URL do którego będzie wysłane żądanie,
- metoda http,
- parametr useToken określający czy wysyłane żądanie podlega autoryzacji,
- ciało zapytania.

```
const apiRequest = async ({
    url,
    method = 'GET',
    useToken = false,
    body = null,
    onSuccess = null,
    onError = null,
    refreshTokens = null,
}) => {
    try {
        const headers = { 'Content-Type': 'application/json' }
        if (useToken) {
            const accessToken = localStorage.getItem('access-token')
            if (accessToken) { headers['Authorization'] = `Bearer ${accessToken}` }
        }
        const response = await fetch(url, {
            method,
            headers: { ...headers },
            body: body ? JSON.stringify(body) : null,
        })
```

Rysunek 4.7. Fragment metody apiRequest znajdującej się w pliku restApiService.jsx

4.4. Konteneryzacja

W celu uproszczenia procesu uruchamiania prezentowanej aplikacji podjęta została decyzja o wdrożeniu konteneryzacji. Zaletą tego rozwiązania jest gwarancja, że każda z warstw aplikacji zawsze uruchamia się w stabilnym, odizolowanym i niezmiennym środowisku. Do konteneryzacji użyto technologii Docker, a do zarządzania procesem uruchomienia i utrzymania aplikacji użyto narzędzia Docker Compose [9]. Na rysunkach 4.8 i 4.9 przedstawiono pliki Dockerfile odpowiedzialne za zarządzanie procesem konteneryzacji odpowiednio warstwy serwerowej oraz klienckiej.

```
FROM openjdk:21-jdk-slim

WORKDIR /server

COPY target/*.jar server.jar

ENTRYPOINT ["java", "-jar", "server.jar"]
```

Rysunek 4.8. Plik Dockerfile warstwy serwera

```
FROM node

WORKDIR /app

COPY package.json /app/

RUN npm install

COPY . /app/

RUN npm run build

EXPOSE 5173

CMD ["npm", "run", "dev"]
```

Rysunek 4.9. Plik Dockerfile warstwy klienta

Plikiem definiującym zasady uruchomienia i utrzymania aplikacji jest plik docker-compose.yml przedstawiony na rysunku 4.10. Warto zwrócić uwagę na ustawienia „restart: always” dla każdego z kontenerów. Powoduje ono, że gdyby z jakiegoś powodu (na przykład w przypadku wystąpienia krytycznego błędu) któryś z kontenerów wyłączył się, narzędzie Docker Compose natychmiastowo z powrotem uruchomi kontener na nowo. Efektem takiego zachowania jest automatyczne utrzymanie ciągłości dostarczanych usług na poziomie programowym. Innym ciekawym aspektem są atrybuty „depends_on” użyte przy kontenerze z warstwą kliencką oraz serwerową. Oznaczają one, że kontener serwerowy nie uruchomi się dopóki nie zostanie zainicjowany kontener z bazą danych, a kontener przechowujący warstwę kliencką nie włączy się dopóki stabilnie nie będzie działała usługa serwera.

```yaml
version: '3.1'

services:
  database:
    container_name: skp-pssql-database
    image: postgres
    ports:
      - "5432:5432"
    restart: always
    environment:
      POSTGRES_PASSWORD: changemeinprod!

  server:
    container_name: skp-spring-server
    image: openjdk:21-jdk-slim
    ports:
      - "8080:8080"
    build:
      context: /server
      dockerfile: Dockerfile
    restart: always
    depends_on:
      - database

  client:
    container_name: skp-react-client
    image: node
    ports:
      - "5173:5173"
    build:
      context: /client
      dockerfile: Dockerfile
    restart: always
    depends_on:
      - server
```

Rysunek 4.10. Zawartość pliku docker-compose.yml

4.5. Funkcjonalność rejestracji konta klienta

Podstawową funkcjonalnością umożliwianą przez aplikację, która jest tematem pracy, jest możliwość rejestracji konta użytkownika w systemie w celu uzyskania dostępu do pozostałych funkcjonalności, których zasoby przypisane są do konkretnego konta. Na tą funkcjonalność składa się formularz do wprowadzania danych rejestrowanego konta przedstawiony na rysunku 4.11. Dane otrzymane z tego formularza przekazywane są do

warstwy serwera za pomocą metody opisanej w podrozdziale 4.3 przy użyciu kodu, którego fragment został przedstawiony na rysunku 4.12. Odebrane przez serwer dane przetwarzane są przez endpoint kontrolera AuthorizationController, którego kod przedstawiony jest na rysunku 4.13.



Rysunek 4.11. Formularz rejestracji nowego konta

```
const url = "http://localhost:8080/api/register"

await apiRequest({
    url,
    method: 'POST',
    body: userData,
    onSuccess: ((status, data) => {
        localStorage.setItem("access-token", data.accessToken)
        localStorage.setItem("refresh-token", data.refreshToken)
        localStorage.setItem("role", data.role)
        toast.success(
            <div>
                Pomyślnie zarejestrowano konto
            </div>,
```

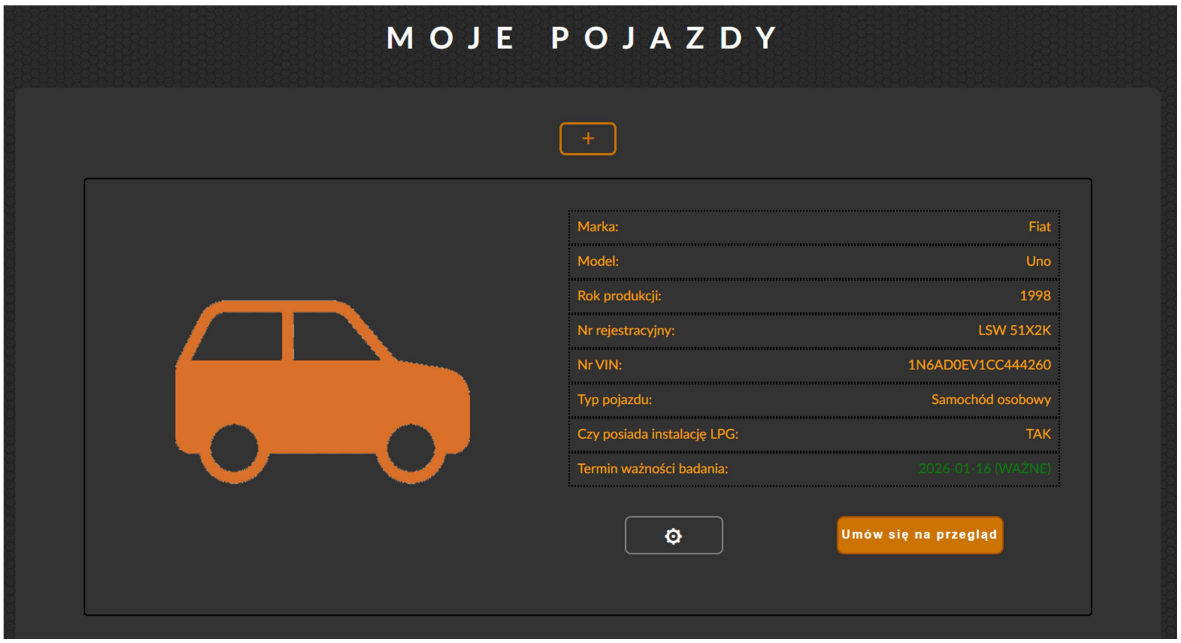
Rysunek 4.12. Fragment kodu metody wysyłającej dane z formularza

```
no usages  Piotrek Bednarski
@PostMapping(path = "/register")
public ResponseEntity<?> register(@RequestBody UserDetailsDto userDetailsDto) {
    Optional<JwtTokenDto> tokenResponse = authenticationService.register(userDetailsDto);
    if(tokenResponse.isPresent()) {
        return new ResponseEntity<>(tokenResponse.get(),HttpStatus.CREATED);
    }
    else {
        return new ResponseEntity<>( body: "Ten email jest juz zajety!",HttpStatus.BAD_REQUEST);
    }
}
```

Rysunek 4.13. Metoda obsługująca endpoint rejestracji za pomocą metody HTTP POST

4.6. Funkcjonalność zarządzania pojazdami użytkownika

Użytkownik zalogowany ma możliwość zarządzania zestawem pojazdów, które są przypisane do jego konta. Funkcjonalność ta obejmuje wszystkie operacje CRUD. Na rysunku 4.14 przedstawiona została lista dodanych pojazdów przez użytkownika.



Rysunek 4.14. Lista dodanych przez użytkownika pojazdów

Na rysunku 4.15 przedstawiony został fragment kodu odpowiedzialnego za poprawne wyświetlane listy pojazdów przypisanych do konkretnego użytkownika pobranej z warstwy serwera.

```
return(
    <div>
        <h2>Moje pojazdy</h2>
        <main>
            <Link className='link-react-router' to="/moje_pojazdy/dodaj_pojazd">
                <div className='plus-add-btn'>&#x2b;</div>
            </Link>
            {data && data.map((vehicle) => {
                const dateStyle = { color: vehicle.validityPeriod < today ? 'red' : 'green' };
                const isExpired = vehicle.validityPeriod < today;
                const statusText = isExpired ? "(NIEWAŻNE)" : "(WAŻNE)";
                return (
                <div key={vehicle.id} className={styles.vehicle}>
                    <div className={styles.photo}>
                        <img className={styles['vehicle-img']} src={vehicle_icon}/>
                    </div>
                    <div className={styles['vehicle-container']}>
                        <div className={styles['vehicle-data']}>
                            <div>Marka:</div><div>{vehicle.brand}</div>
                        </div>

                        <div className={styles['vehicle-data']}>
                            <div>Model:</div><div>{vehicle.model}</div>
                        </div>
```

Rysunek 4.15. Fragment kodu HTML w komponencie MojePojazdy.jsx

Kontroler VehicleController w warstwie serwerowej aplikacji zawiera obsługę wyżej wspomnianych operacji CRUD. Na rysunku 4.16 przedstawiono kod endpointa odpowiedzialnego za przesyłanie do warstwy klienta listy pojazdów pobranych z warstwy usług w kontekście konkretnego autoryzowanego użytkownika, którego nazwa przechowywana jest w SecurityContextHolder.

```
@RestController
@RequestMapping(path = "/api")
@RequiredArgsConstructor
public class VehicleController {

    5 usages
    private final VehicleService vehicleService;

    no usages  Piotrek Bednarski
    @GetMapping(path = "/vehicles")
    public ResponseEntity<?> getAllUsersVehicles() {
        String userName = SecurityContextHolder.getContext().getAuthentication().getName();
        return new ResponseEntity<>(vehicleService.findAll(userName),HttpStatus.OK);
    }
```

Rysunek 4.16. Fragment kontrolera VehicleController

4.7. Funkcjonalność zmiany hasła konta w systemie

Ze względu na bezpieczeństwo podjęto decyzję o wystawieniu funkcjonalności zmiany hasła. Rysunek 4.17 przedstawia formularz zmiany hasła użytkownika zalogowanego.



Rysunek 4.17. Formularz zmiany hasła

Powyższy formularz obsługiwany jest przez kod warstwy klienckiej, którego fragment przedstawiony jest na rysunku 4.18.

```
const handleEditPassword = async (e) => {
    e.preventDefault()

    if(window.confirm('Czy na pewno chcesz zmienić hasło?')){
        if(validPassword(editPwd.newPassword)) {
            const validNewPassword = editPwd.newPassword === editPwd.confirmNewPassword
            if(validNewPassword){
                const passwords = {
                    oldPassword: editPwd.oldPassword,
                    newPassword: editPwd.newPassword
                }
                const url = "http://localhost:8080/api/user/my-account/change-password"
                await apiRequest({
                    url,
                    useToken: true,
                    method: 'PATCH',
                    body: passwords,
                    onSuccess: ((status, data) => {
                        toast.success(
                            <div>
                                Twoje hasło zostało zmienione
                            </div>,
```

Rysunek 4.18. Metoda wysyłająca zapytanie aktualizujące do warstwy serwera

Po stronie serwera opisywana funkcjonalność obsługiwana jest przez kontroler UserController. Na rysunku 4.19 zaprezentowany jest kod endpointa obsługującego zmianę hasła.

```
no usages  Piotrek Bednarski
@PatchMapping(path = "/my-account/change-password")
public ResponseEntity<?> changePassword(
        @RequestBody PasswordChangeDto passwordChangeDto
) {
    if(
            passwordChangeDto == null
            || passwordChangeDto.getNewPassword() == null
            || passwordChangeDto.getOldPassword() == null
    ) return new ResponseEntity<>( body: "Nie otrzymano danych!",HttpStatus.BAD_REQUEST);
    String userContext = SecurityContextHolder.getContext().getAuthentication().getName();
    userService.changePassword(userContext, passwordChangeDto);
    return new ResponseEntity<>( body: "Haslo zostalo zmienione!",HttpStatus.OK);
}
```

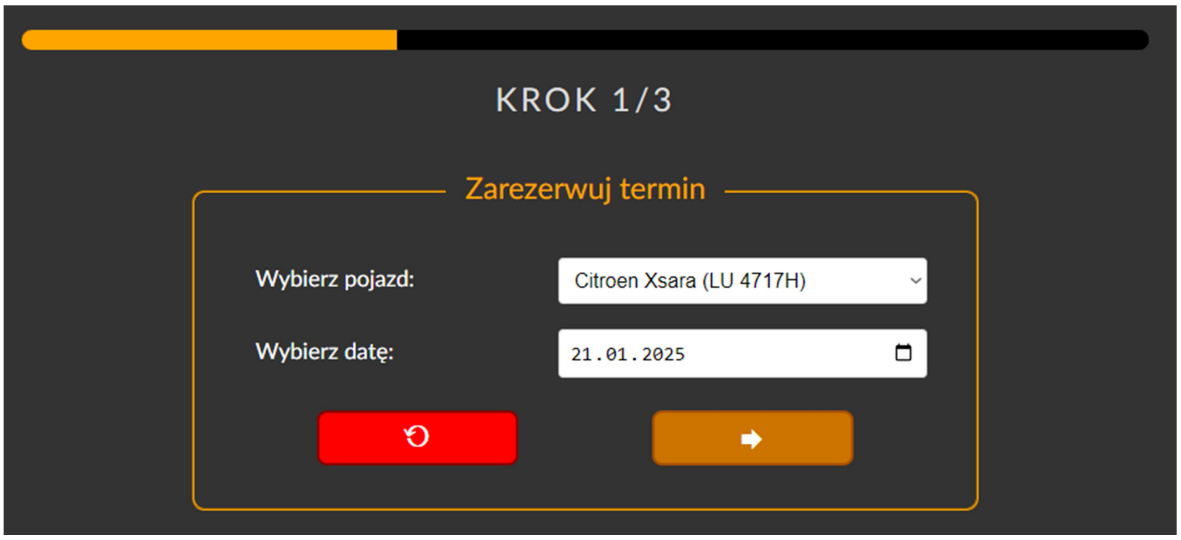
Rysunek 4.19. Metoda obsługująca metodę PATCH

4.8. Funkcjonalność umawiania się na badanie techniczne

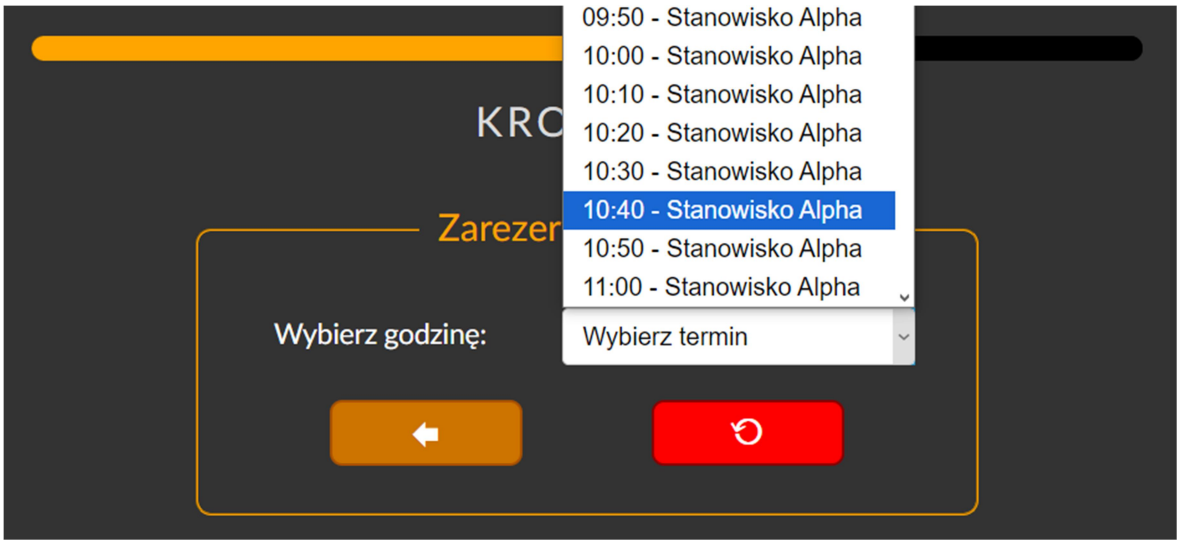
Kluczową funkcjonalnością opisywanego systemu jest możliwość umawiania się na przegląd przez klienta. Proces ten podzielony jest na 3 kroki:

- wybieranie pojazdu i daty,
- wybranie dostępnej godziny i stanowiska,
- potwierdzenie rezerwacji.

Na rysunkach 4.20-22 przedstawiono kolejne kroki omawianej funkcjonalności.



Rysunek 4.20. Widok wyboru pojazdu i daty



Rysunek 4.21. Widok wyboru godziny i stanowiska



Rysunek 4.22. Widok potwierdzania rezerwacji

Na rysunku 4.23 przedstawiono kod warstwy klienta pobierający z serwera informację o dostępnych godzinach i stanowiskach w zależności od specyfikacji wybranego pojazdu oraz wybranej daty.

```
const getFreeTerms = async () => {
    const url = `http://localhost:8080/api/user/inspections/${data.vehicleId}/${data.date}`
    await apiRequest({
        url,
        useToken: true,
        onSuccess: ((status, data) => {
            setFreeTerms(data.hours)
            setSelectedVehicle(data.vehicle)
            setStep(2)
        }),
        onError: ((status, data) => {
            toast.error(data, {
                autoClose: 3000,
            })
        }),
        refreshTokens,
    })
}
```

Rysunek 4.23. Kod metody pobierającej dane o wolnych terminach

Rysunek 4.24 pokazuje kod warstwy klienta odpowiedzialny za potwierdzanie rezerwacji z wybranymi przez użytkownika danymi.

```
const handleSubmit = async (e) => {
    e.preventDefault()

    if(window.confirm('Czy na pewno chcesz zarezerwować ten termin?\nWyrażając zgodę
        const reservationData = {
            vehicleId: data.vehicleId,
            date: data.date,
            time: data.time,
            standId: data.standId,
        }

        const url = "http://localhost:8080/api/user/inspections"

        await apiRequest({
            url,
            useToken: true,
            method: 'POST',
            body: reservationData,
            onSuccess: ((status, data) => {
                toast.success(
                    <div>
                        Pomyślnie zarezerwowano termin badania technicznego.
                    </div>,
                    {
                    onOpen: () => setIsBlocked(true),
                    onClose: () => {
                        window.location.assign('/moje_rezerwacje')
                        setIsBlocked(false)
                    },
```

Rysunek 4.24. Kod metody wysyłającej informację o rezerwacji do warstwy serwera

Po stronie serwera funkcjonalność tą wystawia kontroler InspectionController. Na rysunku 4.25 przedstawiono kod endpointa służącego do wyszukiwania wolnych terminów i stanowisk. Warto zwrócić uwagę na wykorzystanie wzorca projektowego Builder budującego zapytanie do warstwy usługowej.

```java
@GetMapping(path = "/inspections/{vehicleId}/{date}")
public ResponseEntity<?> findFreeHours(
        @PathVariable("vehicleId") Long vehicleId,
        @PathVariable("date") String date
) {
    InspectionPreflightDto data = InspectionPreflightDto
            .builder()
            .vehicleId(vehicleId)
            .date(date)
            .build();
    InspectionPreflightResponseDto freeHours = service.findFreeHours(data);
    return new ResponseEntity<>(freeHours, HttpStatus.OK);
}
```

Rysunek 4.25. Endpoint obsługujący metodę HTTP GET zwracający wolne terminy

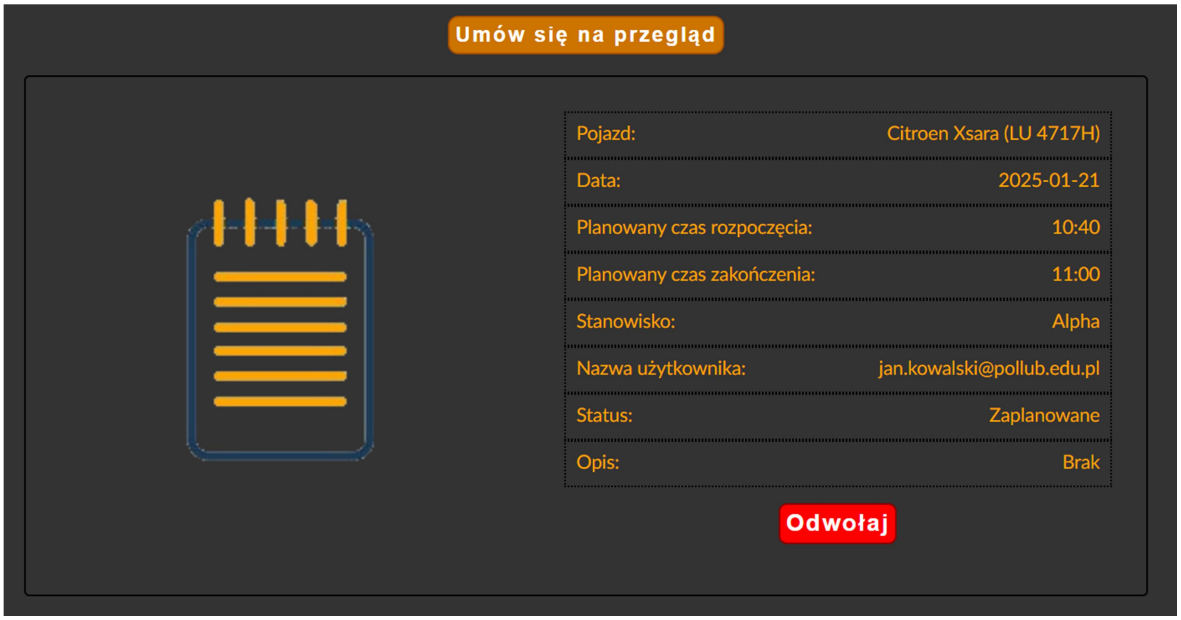
Funkcjonalność tworzenia rezerwacji jest wystawiana przez endpoint, którego kod przedstawiono na rysunku 4.26.

```java
no usages  Piotrek Bednarski
@PostMapping(path = "/inspections")
public ResponseEntity<?> reserveInspection(
        @RequestBody InspectionRequestDto data
) {
    String email = SecurityContextHolder.getContext().getAuthentication().getName();
    Optional<InspectionDetailsDto> createdInspection = service.createInspection(email, data);
    if(createdInspection.isPresent())
        return new ResponseEntity<>(createdInspection.get(),HttpStatus.CREATED);
    else
        return new ResponseEntity<>( body: "Ups! Cos poszlo nie tak!", HttpStatus.INTERNAL_SERVER_ERROR);
}
```

Rysunek 4.26. Endpoint obsługujący metodę HTTP POST służący do tworzenia rezerwacji w systemie

4.9. Funkcjonalność odwoływania rezerwacji

W systemie przewidziano również sytuację w której użytkownik będzie miał potrzebę odwołania zarezerwowanego terminu badania. Służy do tego formularz przedstawiony na rysunku 4.27.



Rysunek 4.27. Wygląd listy rezerwacji użytkownika z przyciskiem służącym do odwoływania rezerwacji

W warstwie klienta użycie tego przycisku wywołuje metodę przedstawioną na rysunku 4.28. Metoda ta wysyła zapytanie typu HTTP DELETE na odpowiedni adres URL do warstwy serwera.

```
const handleDeleteInspection = async (e, id) => {
    e.preventDefault()
    if(window.confirm('Czy na pewno chcesz odwołać tą rezerwację?')){
        const url = "http://localhost:8080/api/user/inspections/"+id
        await apiRequest({
            url,
            useToken: true,
            method: 'DELETE',
            onSuccess: ((status, data) => {
                toast.success(
                    <div>
                        Pomyślnie odwołano rezerwację:<br/>
                        {data.vehicle.brand} {data.vehicle.model}<br/>
                        {data.vehicle.registrationNumber}
                    </div>,
```

Rysunek 4.28. Kod metody obsługującej przycisk odwołujący rezerwację

Po stronie serwera opisana funkcjonalność wystawiana jest przez kontroler InspectionController. Na rysunku 4.29 przedstawiono kod endpointa obsługującego zapytania odwołujące rezerwację

```
no usages   Piotrek Bednarski
@DeleteMapping(path = "/inspections/{id}")
public ResponseEntity<?> deleteInspection(
        @PathVariable("id") Long id
) {
    Optional<InspectionDetailsDto> deletedInspection = service.delete(id);
    if(deletedInspection.isPresent())
        return new ResponseEntity<>(deletedInspection.get(),HttpStatus.OK);
    else return new ResponseEntity<>( body: "Przeglad o takim ID nie istnieje!",HttpStatus.NOT_FOUND);
}
```

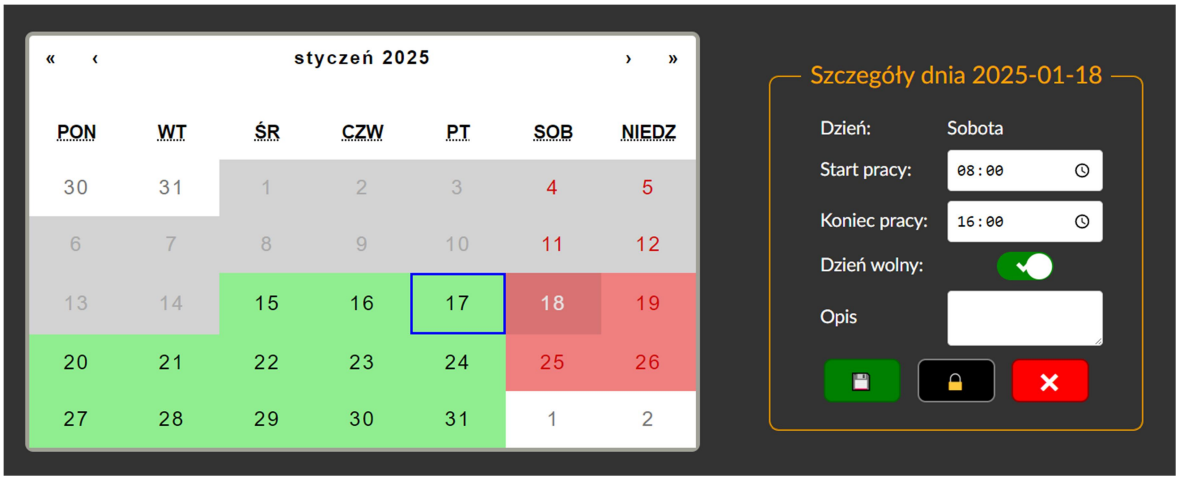
Rysunek 4.29. Endpoint realizujący usuwanie rezerwacji

4.10. Funkcjonalność panelu administratora: Zarządzanie kalendarzem pracy

Administrator systemu ma możliwość aktywacji oraz dezaktywacji zakresu dat w celu zarządzania możliwością rezerwacji badań technicznych przez użytkowników z określonym wyprzedzeniem. Formularz zarządzania kalendarzem pracy umożliwia konfigurację konkretnych dat ze względu na:

- aktywację i dezaktywację zakresu dat lub konkretnego dnia,
- ustalanie godzin pracy konkretnego dnia,
- ustawianie konkretnych dat jako dni wolne.

Te funkcjonalności zostały zawarte w formularzu przedstawionym na rysunkach 4.30 i 4.31 z użyciem komponentu zaimportowanego z biblioteki React Calendar, którego użycie zostało zaprezentowane na rysunku 4.32.



Rysunek 4.30. Zarządzanie poszczególnymi datami w kalendarzu pracy

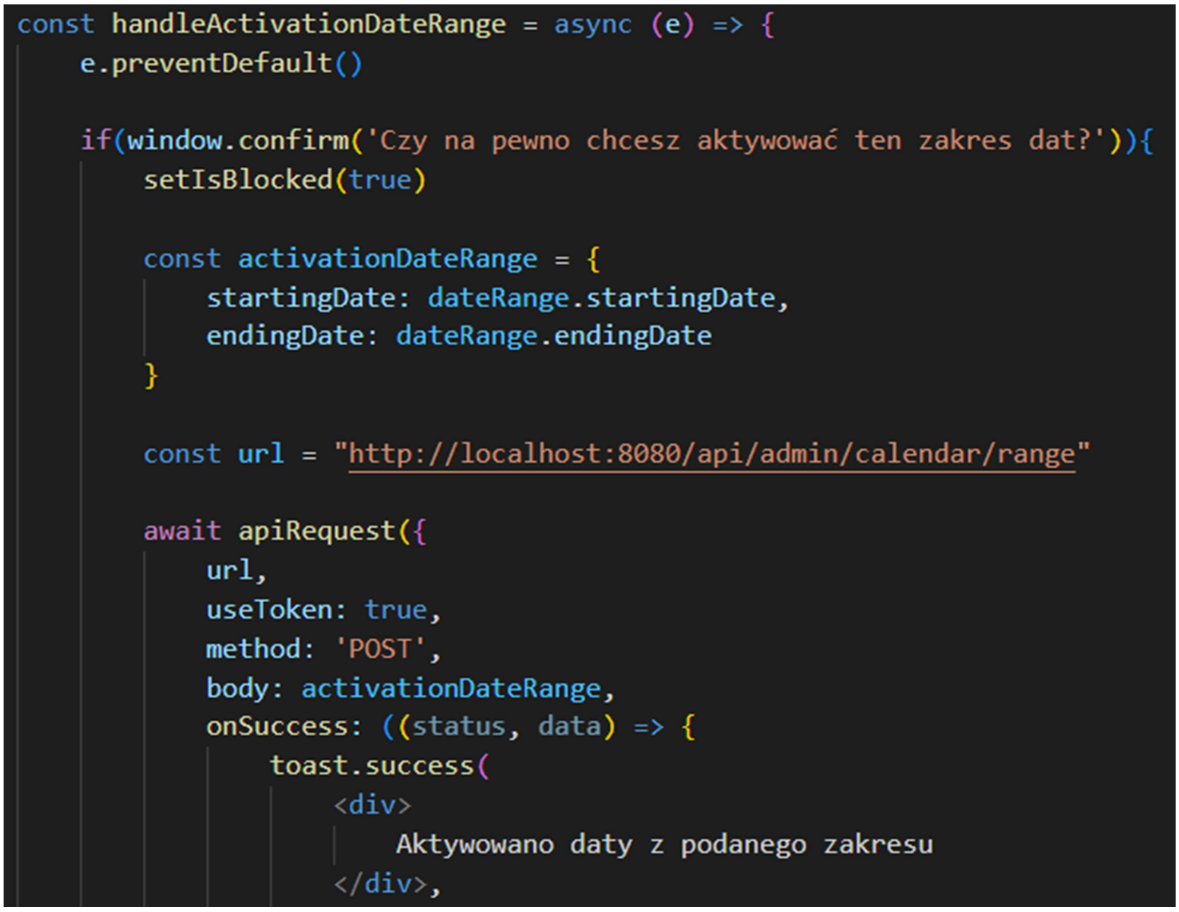


Rysunek 4.31. Aktywacja lub dezaktywacja dat z wybranego zakresu

```
<main>
    <div className='calendar-content'>
        <div className='calendar-container'>
            <Calendar
                value={calendarDate}
                onChange={handleDateChange}
                onActiveStartDateChange={handleActiveStartDateChange}
                tileClassName={tileClassName}
            />
        </div>
```

Rysunek 4.32. Komponent React Calendar użyty w kodzie HTML

Kod odpowiedzialny za wysłanie zapytania aktywującego zakres dat w kalendarzu w warstwie klienckiej został przedstawiony na rysunku 4.33.



Rysunek 4.33. Fragment metody wysyłającej zapytanie aktywujące zakres dat

Po stronie serwera zapytanie to przyjmowane jest przez kontroler CalendarController. Na rysunku 4.34 pokazany jest kod endpointa aktywującego zakres dat. Warto zwrócić uwagę na adnotację @PreAuthorize oznaczającą, że tylko administrator ma dostęp do tej funkcjonalności. Jest to wykorzystanie mechanizmu method-level security pakietu Spring Security



Rysunek 4.34. Endpoint do zarządzania zakresem dat z użyciem method-level security

5. Testy aplikacji

Testowanie aplikacji jest niezwykle istotnym procesem wytwarzania oprogramowania. Testy programowe mają za zadanie weryfikację czy napisany fragment kodu działa poprawnie i czy realizuje zadaną mu funkcjonalność. Dzięki mechanizmom testów programista ma możliwość obserwacji reakcji testowanych funkcji, metod lub obiektów na określone dane wejściowe. Test zawiera dane wejściowe oraz zestaw danych lub zachowań oczekiwanych, definiowanych na przykład poprzez proces asercji

5.1. Podział testów ze względu na przeznaczenie

Wyróżniamy trzy główne rodzaje testów:

- testy jednostkowe,
- testy integracyjne,
- testy funkcjonalne.

Testy jednostkowe

Testy jednostkowe mają za zadanie przetestowanie poprawności działania pojedynczego bloku kodu. Służą one do weryfikacji czy zaprogramowany blok kodu w postaci funkcji, metody lub klasy poprawnie reaguje na określone zestawy danych wejściowych. W tego rodzaju testach często używane są obiekty imitacji (mocki) naśladujące zachowania zależności niezbędnych do funkcjonowania testowanego bloku kodu. Ogólne przygotowanie testów jednostkowych polega na utworzeniu tych obiektów imitacji, definicji ich zachowań, a następnie utworzenie testowanego obiektu poprzez wstrzyknięcie do jego konstruktora stworzonych „mocków”. Tak przygotowany obiekt testowy poddajemy procesom takim jak weryfikacja czy asercja.

Testy integracyjne

Testy integracyjne w przeciwieństwie do testów jednostkowych nie skupiają się na testowaniu działania jednego fragmentu kodu wyjętego z kontekstu komunikacji z innymi blokami funkcjonalnymi, a na testowaniu poprawnej komunikacji pomiędzy warstwami testowanej aplikacji. Testowanie to ma na celu sprawdzenie poprawności działania pojedynczej funkcjonalności, na przykład czy konkretny endpoint zachowuje się w przewidywany dla niego sposób i czy poprawnie realizuje funkcjonalność jaka jest mu przypisana.

Testy funkcjonalne

Testy funkcjonalne mają na celu symulację działania całej testowanej aplikacji w środowisku maksymalnie zbliżonym lub takim samym jak środowisko produkcyjne. Dzięki tym testom możliwe jest stwierdzenie czy aplikacja spełnia założenia przewidywane w wymaganiach funkcjonalnych, a nawet w niektórych wymaganiach niefunkcjonalnych. Jest to najczęściej ostatni etap procesu testowania aplikacji przed wdrożeniem jej w środowisku produkcyjnym. W praktyce w środowiskach biznesowych tego rodzaju testy są nierzadko pomijane ze względu na czasochłonność i zużycie zasobów programistycznych.

5.2. Testy warstwy serwera projektowanego systemu

Na przykładzie projektowanej aplikacji będącej tematem pracy zaprezentowane zostały testy integracyjne. Są to testy klas kontrolerów wystawiających funkcjonalności rejestracji i logowania, gdyż są to kluczowe z punktu widzenia systemu elementy, bez których nie ma możliwości uzyskania dostępu do pełnej zawartości funkcjonalnej aplikacji. Testy zostały wykonane z pakietem „spring-boot-starter-test" przy pomocy frameworka JUnit przy użyciu mechanizmu MockMvc. Mechanizm ten pozwala na symulowanie warstwy klienckiej poprzez możliwość wysyłania zapytań HTTP na testowany endpoint z poziomu kodu języka Java. Testy integracyjne uruchamiają się w testowej wersji aplikacji, która różni się od produkcyjnej wersji aplikacji tym, że działa ona nie na rzeczywistej produkcyjnej bazie danych PostgreSQL, a na testowej bazie danych H2 tworzonej w pamięci RAM.

Test poprawnej rejestracji nowego klienta w systemie

Na rysunku 5.1 zaprezentowano test integracyjny mający na celu sprawdzenie czy endpoint służący do tworzenia nowego konta klienta w systemie pomyślnie zwraca kod HTTP 201 CREATED w przypadku poprawnego przebiegu procesu rejestracji.

```
no usages   Piotrek Bednarski
@Test
public void testThatRegisterSuccessfullyReturnsHttp201Created() throws Exception {
    UserDetailsDto testUser = TestDataUtil.createTestUserDetailsDto();
    String registerJson = objectMapper.writeValueAsString(testUser);
    mockMvc.perform(
            MockMvcRequestBuilders.post( uriTemplate: "/api/register")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(registerJson)
    ).andExpect(
            MockMvcResultMatchers.status().isCreated()
    );
}
```

Rysunek 5.1. Test integracyjny weryfikujący poprawne tworzenie konta klienta

Test weryfikujący jednoczesne logowanie przy zarejestrowaniu konta

Na rysunku 5.2 przedstawiony jest test integracyjny weryfikujący czy pomyślne rejestrowanie konta w systemie automatycznie zaloguje klienta na utworzone właśnie konto. Warto zwrócić uwagę na wartości oczekiwane w ciele zwracanej odpowiedzi HTTP. Fakt istnienia w niej tokenów „accessToken", „refreshToken" i „role" tożsama jest z zalogowaniem użytkownika.

```
no usages   Piotrek Bednarski
@Test
public void testThatRegisterUserAutomaticallyLogInCreatedUser() throws Exception {
    UserDetailsDto testUser = TestDataUtil.createTestUserDetailsDto();
    String registerJson = objectMapper.writeValueAsString(testUser);
    mockMvc.perform(
            MockMvcRequestBuilders.post( uriTemplate: "/api/register")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(registerJson)
    ).andExpect(
            MockMvcResultMatchers.jsonPath( expression: "$.accessToken").isNotEmpty()
    ).andExpect(
            MockMvcResultMatchers.jsonPath( expression: "$.refreshToken").isNotEmpty()
    ).andExpect(
            MockMvcResultMatchers.jsonPath( expression: "$.role").value( expectedValue: "ROLE_CLIENT")
    );
}
```

Rysunek 5.2. Test integracyjny automatycznego logowania na zarejestrowane konto

Test weryfikujący status pomyślnego logowania na istniejące konto

Na rysunku 5.3 widoczny jest kod testu integracyjnego mającego na celu weryfikację czy pomyślne logowanie na istniejące konto zwraca odpowiedź HTTP 200 OK.

```
no usages   Piotrek Bednarski
@Test
public void testThatLoginOnExistingAccountSuccessfullyReturns200Ok() throws Exception {
    UserDetailsDto testUser = TestDataUtil.createTestUserDetailsDto();
    UserDto loginRequest = TestDataUtil.createTestLoginRequest(testUser);
    String loginRequestJson = objectMapper.writeValueAsString(loginRequest);
    service.register(testUser);
    mockMvc.perform(
            MockMvcRequestBuilders.post( uriTemplate: "/api/login")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(loginRequestJson)
    ).andExpect(
            MockMvcResultMatchers.status().isOk()
    );
}
```

Rysunek 5.3. Test integracyjny pomyślnego zalogowania do konta

Test sprawdzający reakcję systemu na niepoprawne dane logowania

Rysunek 5.4 przedstawia metodę testującą czy niepoprawne logowanie pomyślnie zwróci status 403 FORBIDDEN.

```
no usages   Piotrek Bednarski
@Test
public void testThatLoginOnNotExistingAccountReturns403Forbidden() throws Exception {
    UserDetailsDto testUser = TestDataUtil.createTestUserDetailsDto();
    UserDto loginRequest = TestDataUtil.createTestLoginRequest(testUser);
    String loginRequestJson = objectMapper.writeValueAsString(loginRequest);
    mockMvc.perform(
            MockMvcRequestBuilders.post( uriTemplate: "/api/login")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(loginRequestJson)
    ).andExpect(
            MockMvcResultMatchers.status().isForbidden()
    );
}
```

Rysunek 5.4. Kod metody testującej zachowanie systemu po niepoprawnej próbie zalogowania

Test rejestracji istniejącego konta

Rysunek 5.5 przedstawia test integracyjny weryfikujący odpowiedź serwera na próbę rejestracji konta na istniejący w bazie danych adres e-mail. Oczekiwane zachowanie testowanego endpointa to odesłanie odpowiedzi z kodem HTTP 400 BAD REQUEST.

```
no usages   Piotrek Bednarski
@Test
public void testThatRegistrationOnExistingEmailReturns400BadRequest() throws Exception {
    UserDetailsDto testUser = TestDataUtil.createTestUserDetailsDto();
    String registerRequestJson = objectMapper.writeValueAsString(testUser);
    service.register(testUser);
    mockMvc.perform(
            MockMvcRequestBuilders.post( uriTemplate: "/api/register")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(registerRequestJson)
    ).andExpect(
            MockMvcResultMatchers.status().isBadRequest()
    );
}
```

Rysunek 5.5. Test integracyjny weryfikujący unikalność adresu e-mail w systemie

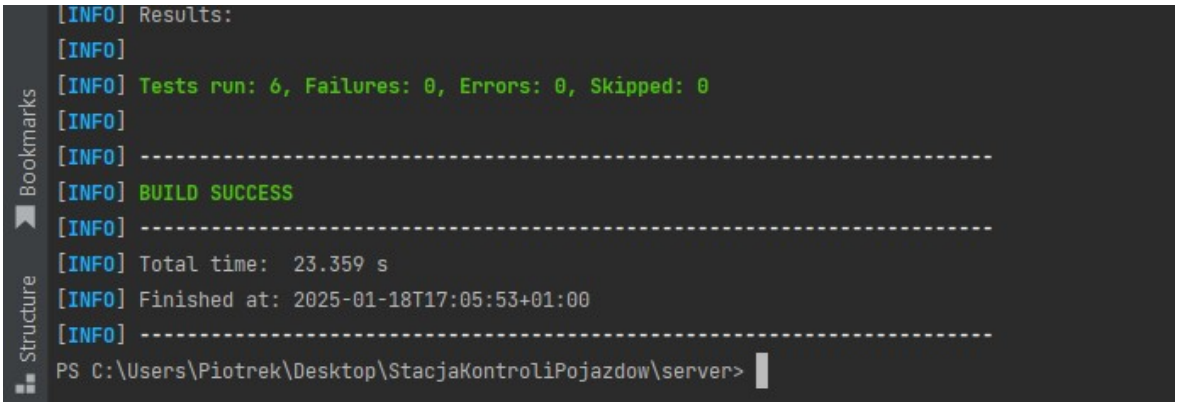
Uruchomienie zaimplementowanych testów integracyjnych

Za pomocą polecenia wrappera buildtool'a Maven:

```
./mvnw clean test
```

- czyszczona jest zawartość poprzedniej kompilacji,
- kompilowana jest aktualna wersja oprogramowania,
- uruchamiane są testy.

Wynikiem uruchomienia tego polecenia jest wyświetlenie statystyk dotyczących ilości wykonanych testów, ilości testów zakończonych niepowodzeniem, ilości błędnych testów oraz ilości testów pominiętych. Wyniki zobrazowane są na rysunku 5.6.



Rysunek 5.6. Statystyki testów aplikacji

6. Wnioski

Założony cel zaprojektowania, implementacji i przetestowania opisywanego w niniejszej pracy systemu informatycznego został w pełni zrealizowany. Wszystkie przewidziane wymagania funkcjonalne i niefunkcjonalne mają odzwierciedlenie w implementacji. System ten jest w pełni sprawny i konkurencyjny z innymi analogicznymi systemami. Można stwierdzić, że Spring Framework razem z warstwą Spring Boot doskonale nadaje się do realizacji API w architekturze REST, a język Java jest bardzo dobrze przystosowany do implementacji architektury serwerowej aplikacji internetowych. Język JavaScript znakomicie współpracuje z kodem HTML oraz CSS rozumianym przez przeglądarki internetowe poprzez format JSX. Wraz z mechanizmami dostarczanymi przez bibliotekę React tworzy on bardzo przyjazne dla programisty środowisko programowania warstw klienckich rozbudowanych aplikacji internetowych. Dzięki temu mogą powstawać przystępne dla użytkowników i konkurencyjne na rynku oprogramowania interfejsy interakcji człowiek-komputer. Docker jako narzędzie konteneryzacji jest niezwykle dobrze udokumentowany i popularny w środowisku producentów oprogramowania. Pokazuje to fakt, iż największe koncerny wystawiające usługi chmurowe nie tylko akceptują to narzędzie konteneryzacji, ale również nierzadko traktują ten format jako zalecany. Z punktu widzenia dalszego możliwego rozwoju niniejszego systemu informatycznego warte rozważenia jest wdrożenie go jako usługa w chmurze obliczeniowej. Można także rozważyć dalszą pracę i ulepszanie kodu systemu w kontekście rozbudowy zestawu realizowanych funkcjonalności takich jak dynamiczne przydzielanie stanowiska do umawianej rezerwacji badania technicznego, archiwizacji danych o terminach odwołanych, wdrożenie powiadomień e-mail i SMS informujących użytkowników o nadchodzącym zarezerwowanym terminie przeglądu, czy przypomnienia o bliskim wygaśnięciu ważności badania technicznego. Wartą rozważenia funkcjonalnością mogła by być możliwość wykonywania oraz przywracania kopii zapasowej bazy danych w celu zredukowania skutków ewentualnej awarii.

W trakcie projektowania niniejszego systemu występowały liczne problemy z oprogramowaniem Visual Paradigm służącym do tworzenia diagramów UML. Z punktu widzenia twórców prezentowanego systemu używanie tego oprogramowania jest trudzące i ogólnie niezalecane.

Bibliografia

[1] Accomazzo A. i inni, Fullstack React, The Complete Guide to ReactJS and Friends, Wydawnictwo Fullstack.io, 2017

[2] Autotest Polska, https://www.autotestpolska.pl/, [11.11.2024]

[3] Centralna Ewidencja Pojazdów i Kierowców, Stacje kontroli pojazdów (SKP) i badania techniczne (BT) w 2022 r., https://www.gov.pl/web/cepik/stacje-kontroli-pojazdow-skp-i-badania-techniczne-bt-w-2022-r, [11.11.2024]

[4] Diagnostyka Rzeszów, https://www.diagnostykarzeszow.pl/, [11.11.2024]

[5] Drejewicz Sz., Zrozumieć BPMN. Modelowanie procesów biznesowych. Wydanie 2 rozszerzone, Wydawnictwo Onepress, 2017

[6] Euromaster, https://www.euromaster.pl/wizyta, [11.11.2024]

[7] Fowler M., UML w kropelce, wersja 2.0, Wydawnictwo LTP, 2005

[8] Główny Urząd Statystyczny, dane z kategorii K8, grupa G239, podgrupa P2420, wskaźnik: samochody osobowe na 1000 ludności, https://bdl.stat.gov.pl/bdl/dane/podgrup/tablica, [11.11.2024]

[9] McKendrick R., Scott G., Docker. Programowanie aplikacji dla zaawansowanych. Wydanie II, Wydawnictwo Helion, 2018

[10] Rozporządzenie Ministra Infrastruktury z dnia 29 września 2004 r. w sprawie wysokości opłat związanych z prowadzeniem stacji kontroli pojazdów oraz przeprowadzaniem badań technicznych pojazdów, Dz.U. 2004 nr 223 poz. 2261, https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU20042232261, [11.11.2024]

[11] Tidwell J., Brewer C., Valencia-Brooks A., Projektowanie interfejsów. Sprawdzone wzorce projektowe. Wydanie III, Wydawnictwo Helion, 2020

[12] Ustawa z dnia 1 lutego 1983 r. Prawo o ruchu drogowym, Dz.U. 1983 nr 6 poz. 35, https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU19830060035, [11.11.2024]

[13] Ustawa z dnia 20 czerwca 1997 r. - Prawo o ruchu drogowym, Dz.U. 1997 nr 98 poz. 602, https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU19970980602, [11.11.2024]

[14] Walls C. Spring w akcji, Wydanie IV, Wydawnictwo Helion, 2015